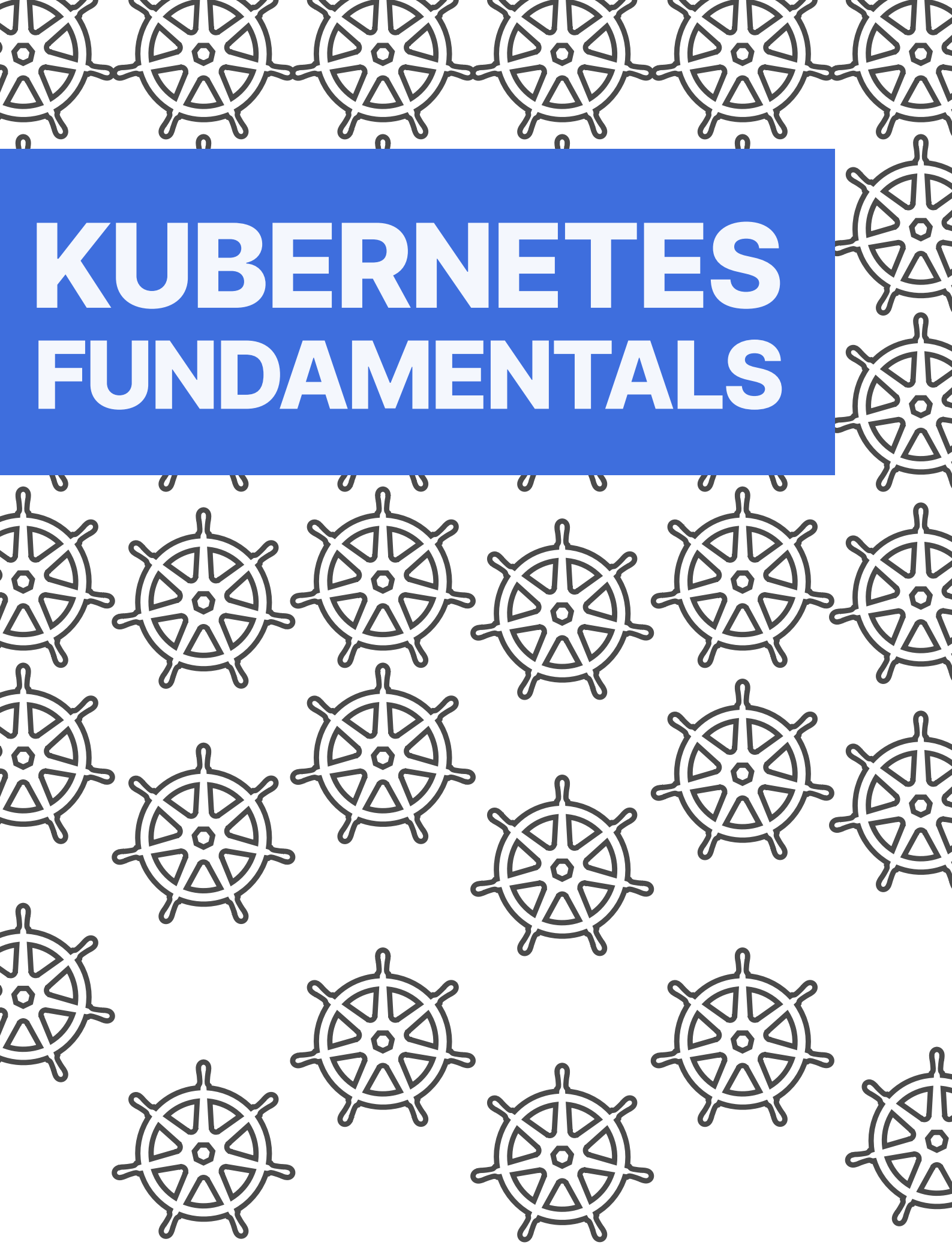
# KUBERNETES FUNDAMENTALS



# Table of contents


**1. Managing containers at scale 6**

Managing containers allocations 10

Managing placements 12

Containers, networking and service discovery 14

**2. Running containers at scale 17**

Datacentre as a computer 18

Kubernetes as a scheduler 24

Kubernetes as an API 27

**3. Deploying applications on Kubernetes 33**

Watching over Pods 39

**4. Creating resources in Kubernetes 42**

YAML maps 45

YAML lists 48

Getting your hands dirty 50

**5. Prerequisites 51**

Windows 10 52

macOS 54

Ubuntu 57

Installing VirtualBox 57

Installing kubectl 58

minikube 58

**6. First deployment 61**

Starting the local cluster 62

Your first deployment 64

Inspecting YAML resources 68

Exposing the application 76

Recap 82

**7. Self-healing and scaling 84**

Practicing chaos engineering 85

Scaling the application 90

Scaling the Service 94

Recap 95

**8. Deploying an app on Kubernetes end-to-end 97**

Creating the application 99

Containerising the application 100

Running the container 104

Creating the image in Minikube 106

Deploying the app in Kubernetes 112

Defining a Service 118

Defining an Ingress 124

Deploying the application 127

Recap 130

**9. Organising resources 131**

Namespaces and (lack of) isolation 138

Recap 139

**10. Practicing deployments 141**

Version 2.0.0 142

Deployment 144

Service 144

Ingress 145

Namespace 145

Version 3.0.0 146

**11. Making sense of Deployment, Service and Ingress 149**

Connecting Deployment and Service 153

Connecting Service and Ingress 158

Recap 162

**12. Why Pods? 163**

Pods as a collection of containers 164

Chapter 1

# Managing containers at scale

Deploying containers is convenient because they conform to a consistent interface.

You don't have to learn how to run a Ruby, Java or .NET application.

As long as the app is packaged as a container, you can run any of the above with `docker run`.

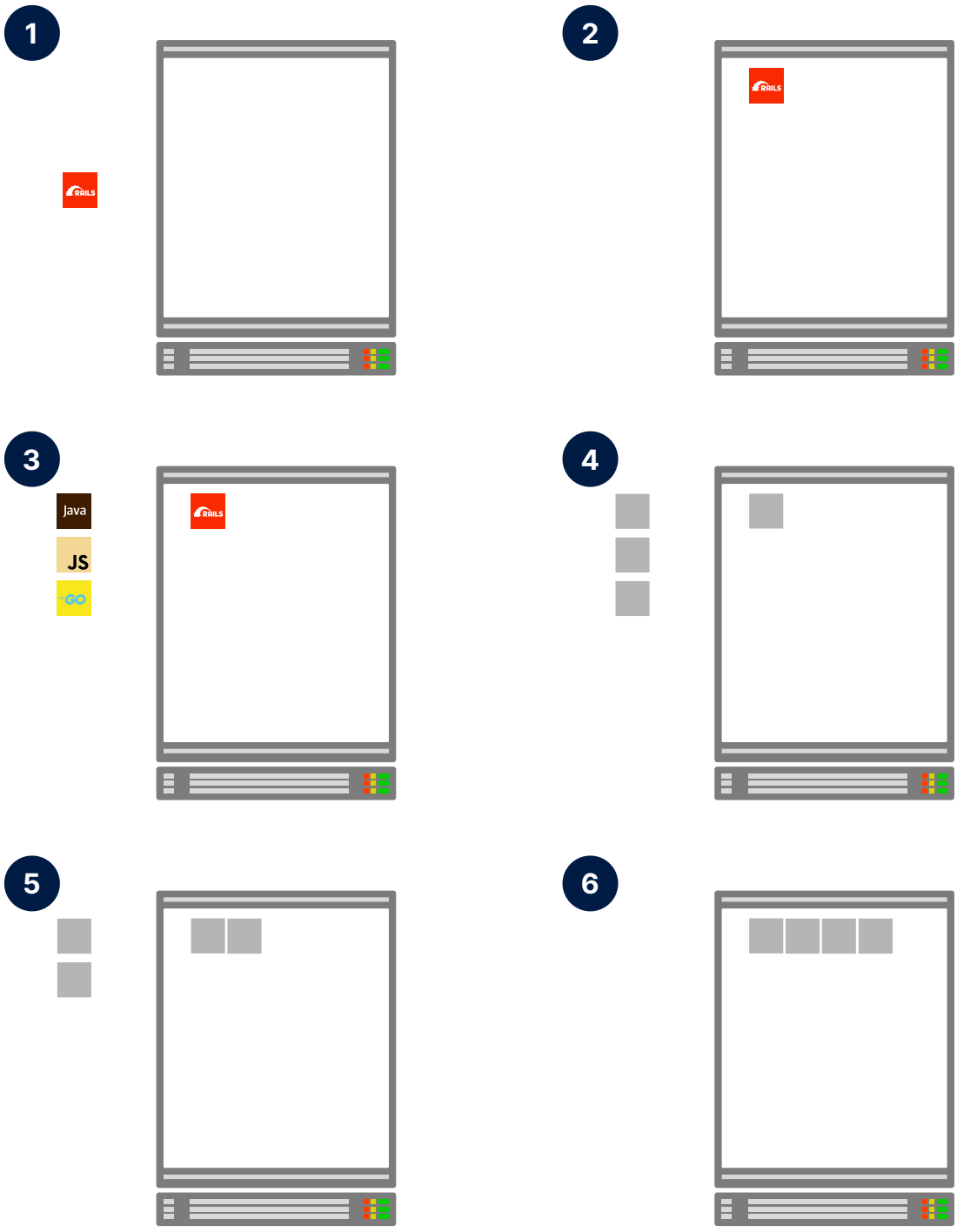



**Fig. 1** In the past, you might have deployed applications such as Ruby on Rails that required an environment to provisioned upfront.

**Fig. 2** The environment wasn't always portable and you had to create it yourself.

**Fig. 3** If you managed more than one app or programming language, you might have faced the challenge of provisioning and keeping environments up to date.

**Fig. 4** Containers are designed to simplify running application in development and production.

**Fig. 5** As long as you can execute `docker run`, you can run containers in your servers without worrying about how to provision an environment for those applications.

**Fig. 6** Containers look all the same from the outside. You can't tell the difference between a Golang and Java application. Both can start with the same `docker run` command.

**Containers are an excellent abstraction if you wish to share resources and isolate processes on the same host.**

*But they don't solve the problem of running processes across several hosts.*

# Managing containers allocations

When you have hundreds if not thousands of containers, you should find a way to run multiple containers on the same server.

And it would be best if you planned for containers to be spread over multiple servers too.

When you can distribute the load across several nodes, you can prevent that a single failure could take down the entire service.

You can also scale your application as you receive more traffic.

*But keeping track of where every container is deployed in your infrastructure is time-consuming and complicated.*




**Fig. 1** When you have a single server, you don't get to choose where your containers run.

**Fig. 2** But as soon as you have more than one computer, you might ask yourself — what should be the best way to launch containers in different servers?

**Fig. 3** Of course, you should try to balance the load. *But manually?*

**Fig. 4** What happens when you have hundreds of servers, and you wish to orchestrate the containers to run in production? *Should you manually place them?* It's not scalable.

*But you could write a program to do it for you.*

An algorithm that can place containers and has a list of all deployed applications and servers.

The algorithm could be smart enough to **pack containers efficiently so to maximise server density but also intelligent enough to minimise disruption if a server becomes unavailable.**

Allocating and balancing containers in your data centre isn't the only challenge, though.

## Managing placements

If you have a diverse set of servers, you want your containers for data-intensive processing — such as machine learning jobs — always to be scheduled on servers with dedicated GPUs.

Similarly, if a container is mounting a volume on a server, you don't want that container to be moved somewhere else because you could lose the data.

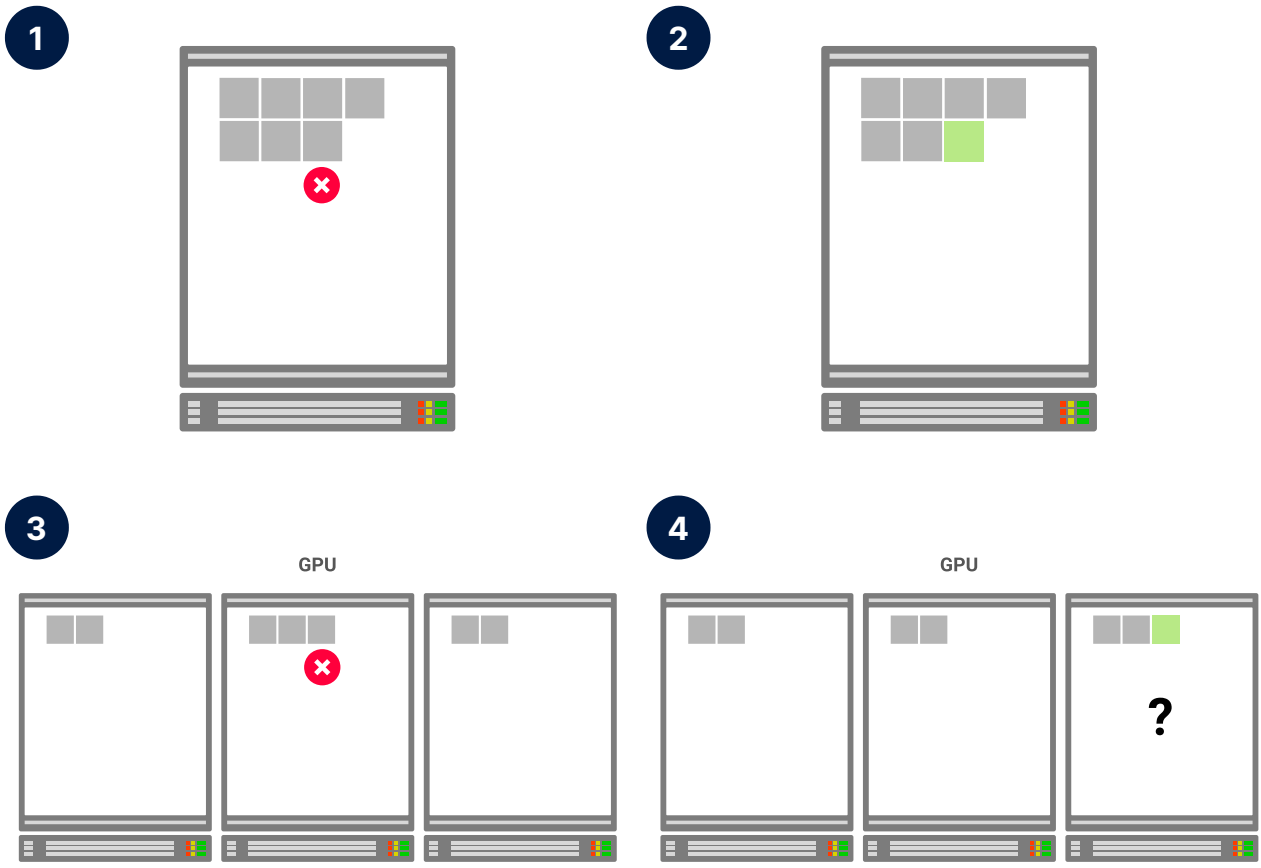



**Fig. 1** When you have a single server, you don't need to worry too much about restarting and moving containers. They all end up on the same server.

**Fig. 2** When you have a single server, you don't need to worry too much about restarting and moving containers. They all end up on the same server.

**Fig. 3** But when you have several compute resources and some of them with particular hardware, *can you keep track of where each container is and their requirements?*

**Fig. 4** You don't want to move a container that requires GPU hardware to a server that can't provide it.

You could tweak your scheduling algorithm and make sure that placements are taken into consideration before launching containers.

The algorithm isn't as simple as you envisioned, but it's doable and promising.

You also have to solve the challenge with networking and service discovery.

## Containers, networking and service discovery

When you run multiple servers, **the containers should be able to connect and exchange messages to each other.**

However, the Docker network doesn't offer a tool out of the box to do that.

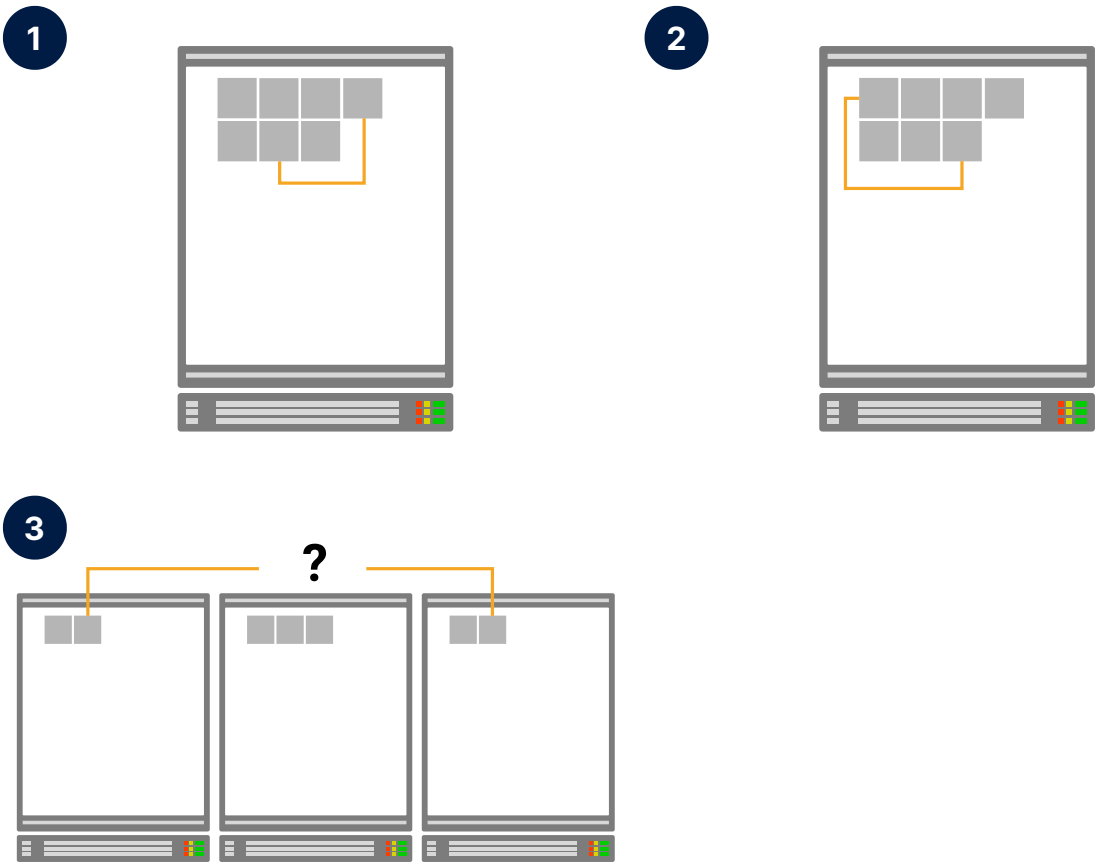



**Fig. 1** When you have a single server, you can use the Docker network to connect the containers.

**Fig. 2** The Docker network offers an easy domain name resolution within the network: the name of the container resolves to its IP address.

**Fig. 3** When you have multiple servers, you can't benefit from the Docker network. You should find a way to connect multiple networks.

**Managing containers on a single server is a different challenge from running containers at scale.**

And Google knows that well.

Chapter 2

# Running containers at scale

Google ran a technology similar to Linux containers and had to find an efficient way to schedule workloads that could span hundreds of thousands of servers.

They didn't want to keep and manually update a long list of containers and servers.

They have servers optimised for CPU workloads as well as GPUs, TPUs, etc. They had to find a way to allocate containers efficiently.

They also had to create a unified network that could span multiple servers and regions.

So they decided to write a platform that can automatically analyse resource utilisation, schedule and deploy containers.

Later on, a few Googlers decided to leave the company and restart the project as an open-source effort.

**And Kubernetes was born.**

You can think of Kubernetes as three things:

1. a single computer that abstracts your data centre
2. a scheduler
3. a unified API layer over your infrastructure

## Datacentre as a computer

**Dealing with a single computer is more straightforward than coordinating deployments across a fleet of servers.**

You noticed it earlier.

Managing placements and allocations, as well as networking and service discovery, were non-problems with a single server.

Also, with a single server, you didn't have to worry about on which server your application runs.

*But how can you use a single server for all of your apps?*

You can't.

But you could create an abstraction.

You could pretend to **make a more powerful computer from merging smaller compute resources**.

That's what Kubernetes does.

Kubernetes embraces the idea of treating your servers as a single unit and abstracts away how individual compute resources operate.

From a collection of three servers, Kubernetes makes a single cluster that behaves like one.

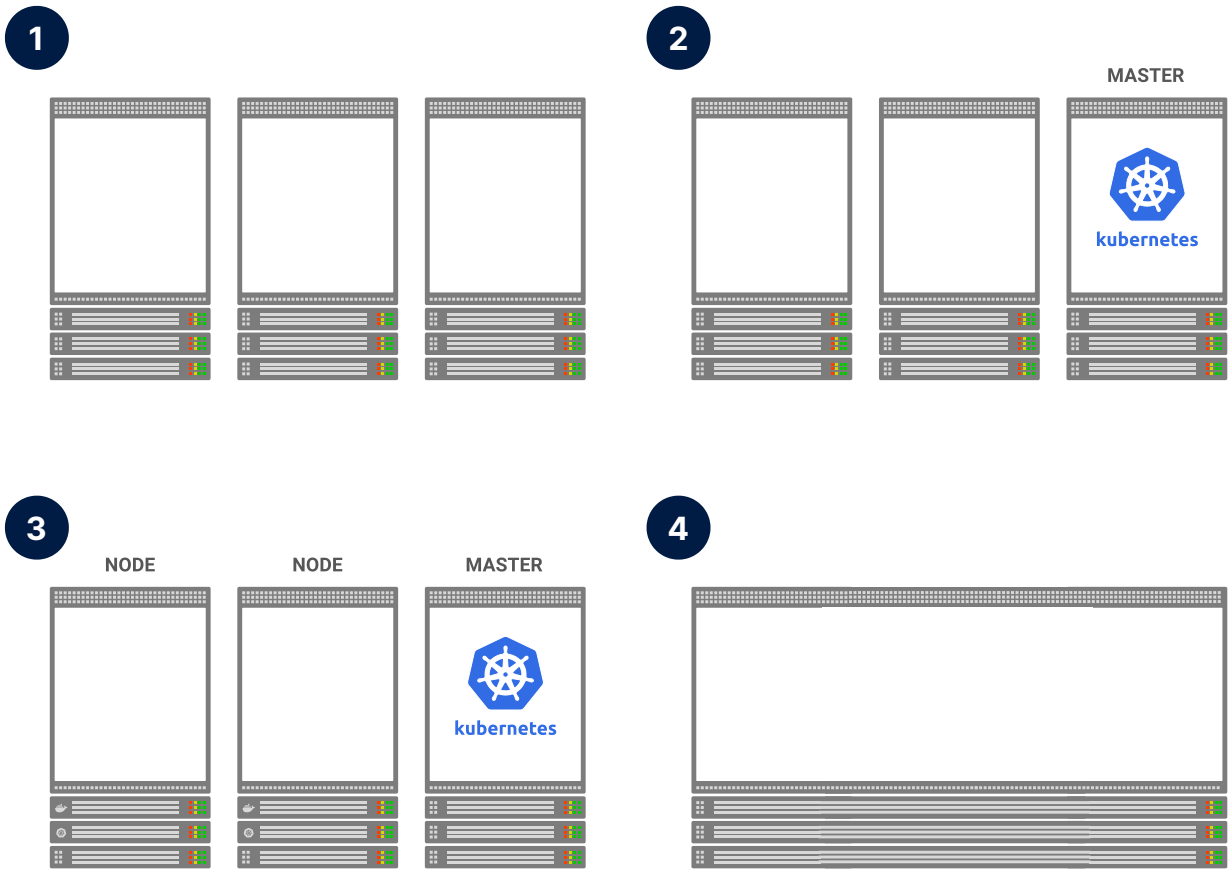



**Fig. 1** Imagine having three compute resources.

**Fig. 2** One of the servers is in charge of being the primary node.

**Fig. 3** The rest of the servers join the master node and form a cluster.

**Fig. 4** The cluster behaves as if the three machines were one.

*What happens when you deploy applications in the cluster?*

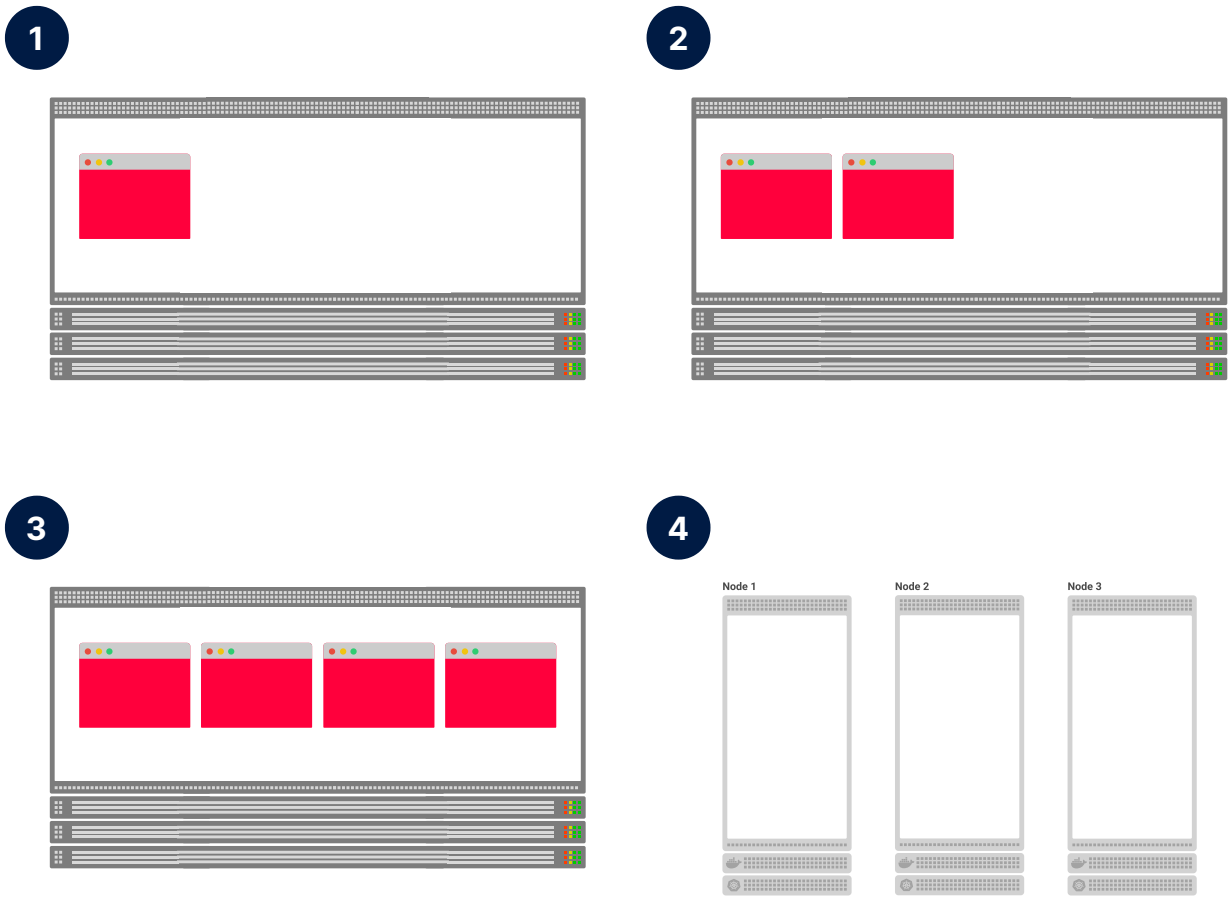



**Fig. 1** When you ask Kubernetes to deploy a few containers, you don't specify where those containers should be placed.

**Fig. 2** You don't fiddle with the networking or register the IP address in the domain name service.

**Fig. 3** The cluster behaves like one, and you feel like you're dealing with a single server.

**Fig. 4** In practice, Kubernetes takes over and decides where each container should be placed in the infrastructure.

**Fig. 5** In practice, Kubernetes takes over and decides where each container should be placed in the infrastructure.

Kubernetes isn't picky.

You can give it any resource that can have memory and CPU and have them join the cluster.

> If you're into embedded systems, there's an active community interested in running Kubernetes cluster on Raspberry PIs.




**Fig. 1** You can have any servers joining the cluster. The size doesn't matter as long as it can offer CPU and memory.

**Fig. 2** When a compute resource joins the cluster, memory and CPU are added together.

**Fig. 3** The cluster has the CPU and memory of all the over servers combined.

**Fig. 4** The cluster has the CPU and memory of all the over servers combined.

**Fig. 5** The cluster has the CPU and memory of all the over servers combined.

CPU and memory for each compute resource are stored in the database and used to decide how to allocate containers.

## Kubernetes as a scheduler

When you deploy an application, Kubernetes identifies the memory requirements for your container and finds the best place to deploy your application.
*You don't decide which server the application is deployed.*
Kubernetes has a scheduler that decides it for you.
The scheduler is optimised to pack containers efficiently:

- initially, it spreads containers over the nodes to maximise resiliency

- as time goes on, the scheduler will try to pack containers in the same host, while still trying to balance resource utilisation

You can think about **Kubernetes scheduler as a skilled Tetris player.**
Docker containers are the blocks; servers are the boards, and Kubernetes is the player.

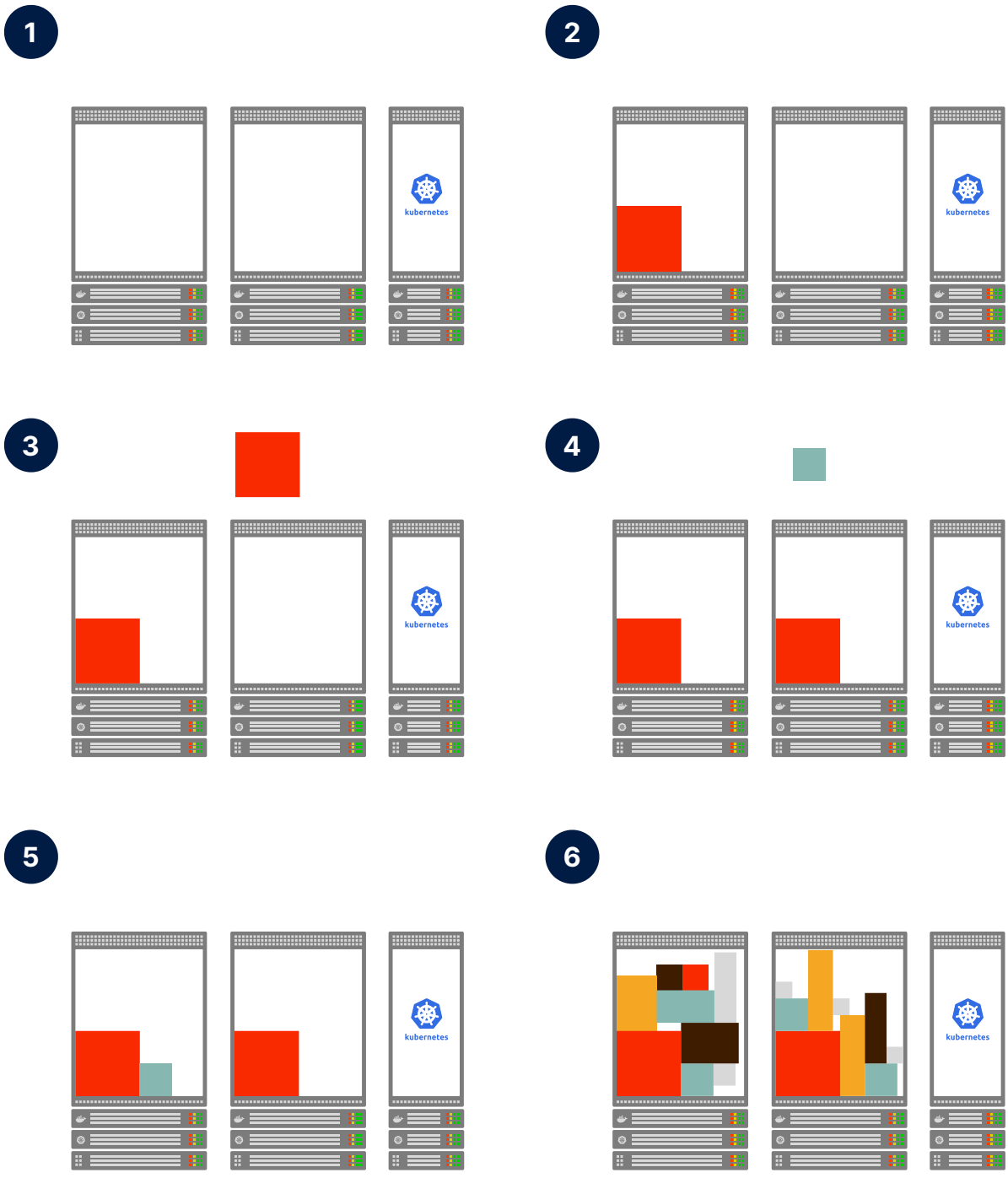



**Fig. 1** Imagine having a Kubernetes cluster with two compute resources.

**Fig. 2** When the cluster receives a request to deploy a container, it scans the worker Nodes and find the best fit.

**Fig. 3** If there're two copies of the same container, Kubernetes deploys the second on the right side. That's the least utilised Node.

**Fig. 4** Another request to deploy a container. The square is smaller because it represents a container with fewer requirements of memory and CPU.

**Fig. 5** Kubernetes decides to place it on the left.

**Fig. 6** Kubernetes is the best tetris player and packs the containers tightly together.

Kubernetes keeps playing the game every time you request a deployment.

## Kubernetes as an API

When you wish to deploy a container in Kubernetes, you end up calling a REST API.

Kubernetes ingests the request and, as soon as the scheduler identifies the right place, a container is deployed.

If you wish to provision a load balancer to route traffic to

your application, Kubernetes exposes a REST API endpoint for that too.

There're REST API endpoints for things such as provisioning storage, autoscaling, managing access and more.

The advantages of everything being an API in Kubernetes are:

- you can create scripts and daemons that **interact with the API programmatically**
- The **API is versioned**; when you upgrade your cluster you can keep using the old API and gradually migrate
- You can install **Kubernetes in any cloud provider or data centre**, and you can leverage the same API

> Starting with Kubernetes 1.10, the Kubernetes API serves an OpenAPI spec. You should take your time and inspect the API.

*You could think of Kubernetes as a layer on top of your infrastructure.*

When you issue a command through a REST API, you don't worry about the underlying infrastructure.

Kubernetes acts as a translator between yourself and your cloud provider (or data centre).

Even if there are differences in implementation between

Azure and Amazon Web Services, you can still rely on Kubernetes to abstract those away.

And since this layer is generic and it can be installed anywhere, you can always take it with you.

Even if you migrate from your on-premise cluster to Amazon Web Services or Azure.

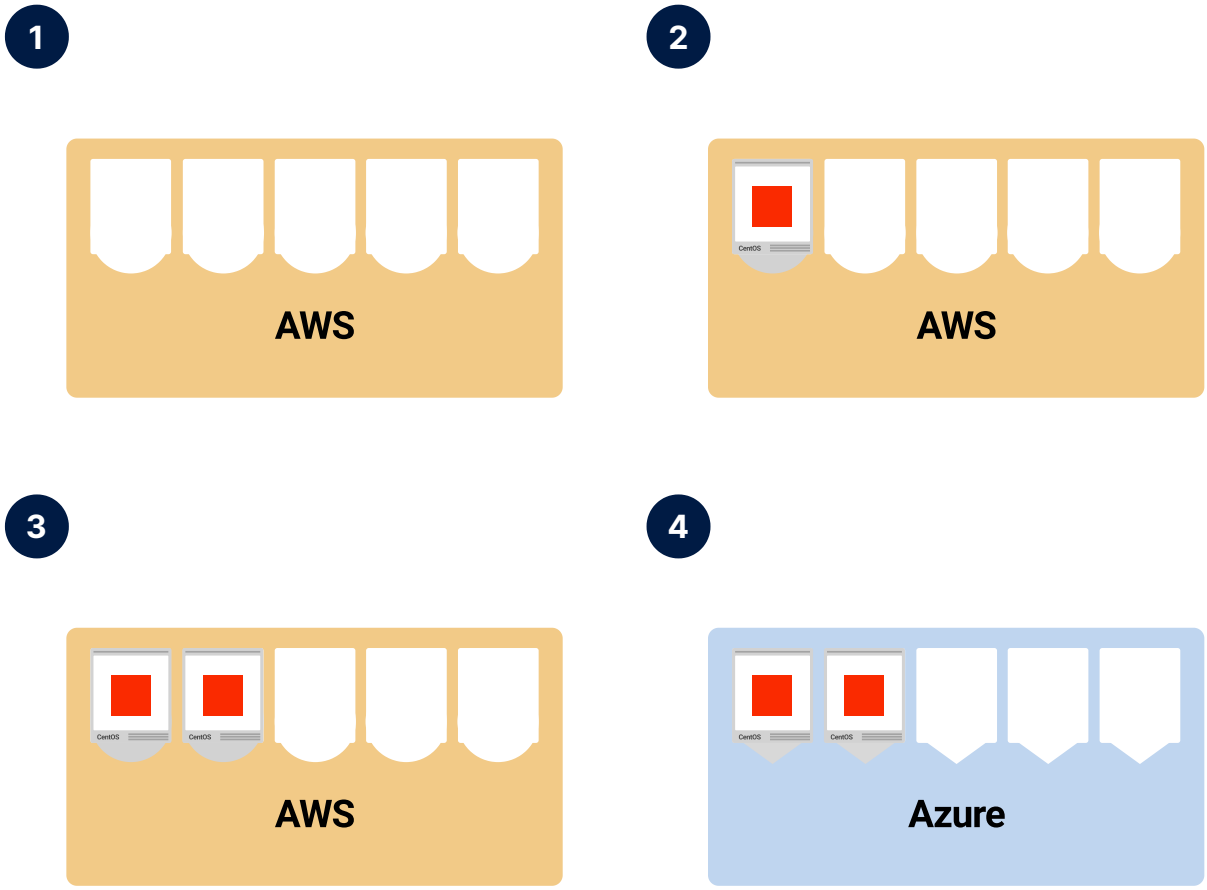



**Fig. 1** In Amazon Web Services you can create virtual machine using the Elastic Compute Cloud (EC2).

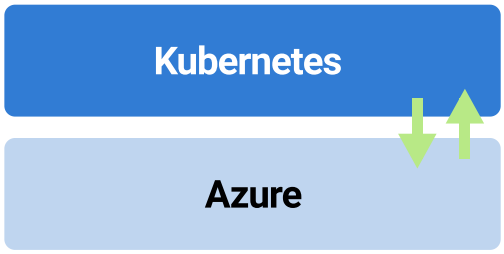
**Fig. 2** The virtual machine are like any other virtual machine.

**Fig. 3** However, Amazon has its secret sauce for its virtual machines. As an example, virtual machines that run in Elastic Compute Cloud use Amazon Machine Images (AMI) as base images.

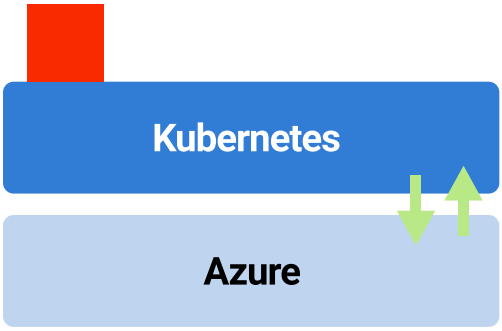
**Fig. 4** If you wish to export the images to use in another cloud provider such as Azure, you can't. Azure uses a different standard. There's limited portability between clouds even if they offer similar services.

*How does Kubernetes solve this challenge?*

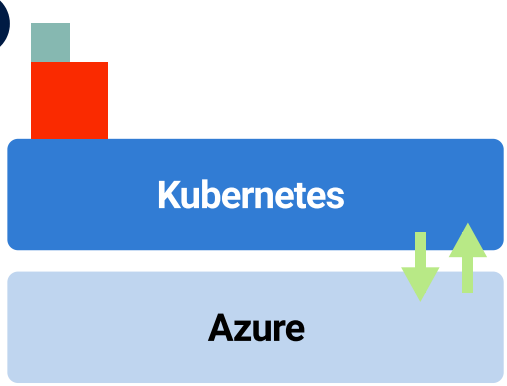
1


Kubernetes
Azure

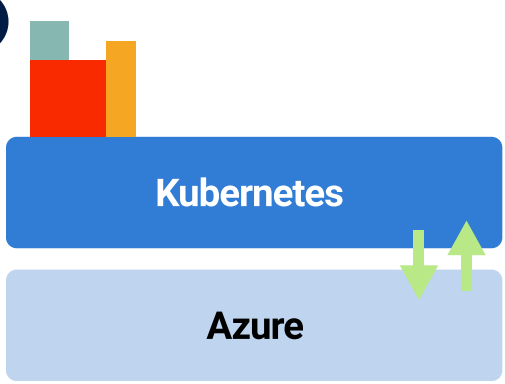
2


Kubernetes
Azure

3


Kubernetes
Azure

4


Kubernetes
Azure

5

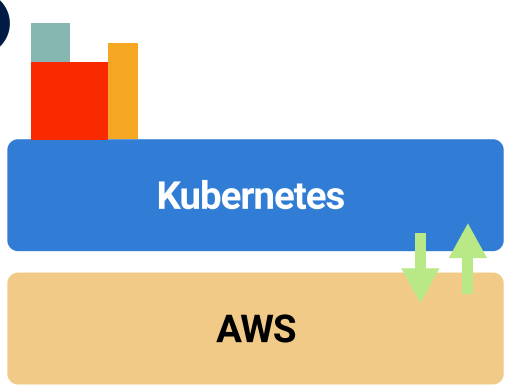

Kubernetes
AWS

**Fig. 1** Kubernetes is designed to abstract the differences between cloud providers and on-premise infrastructure. You make requests to Kubernetes, and Kubernetes translate those into specific instructions for the underlying compute resources.

**Fig. 2** When you deploy a container, Kubernetes takes care of the complexity of having the right resources and finding the right server.

**Fig. 3** When you deploy a container, Kubernetes takes care of the complexity of having the right resources and finding the right server.

**Fig. 4** When you deploy a container, Kubernetes takes care of the complexity of having the right resources and finding the right server.

**Fig. 5** When you realise that Azure is more expensive than Amazon Web Services for your tasks, you can swap the underlying layer and still deploy applications as if nothing happened.

Now that you know how Kubernetes works, you're ready for your first deployment.

Chapter 3

# Deploying applications on Kubernetes

You decide upon deploying an application to Kubernetes that displays a static page with a red background.

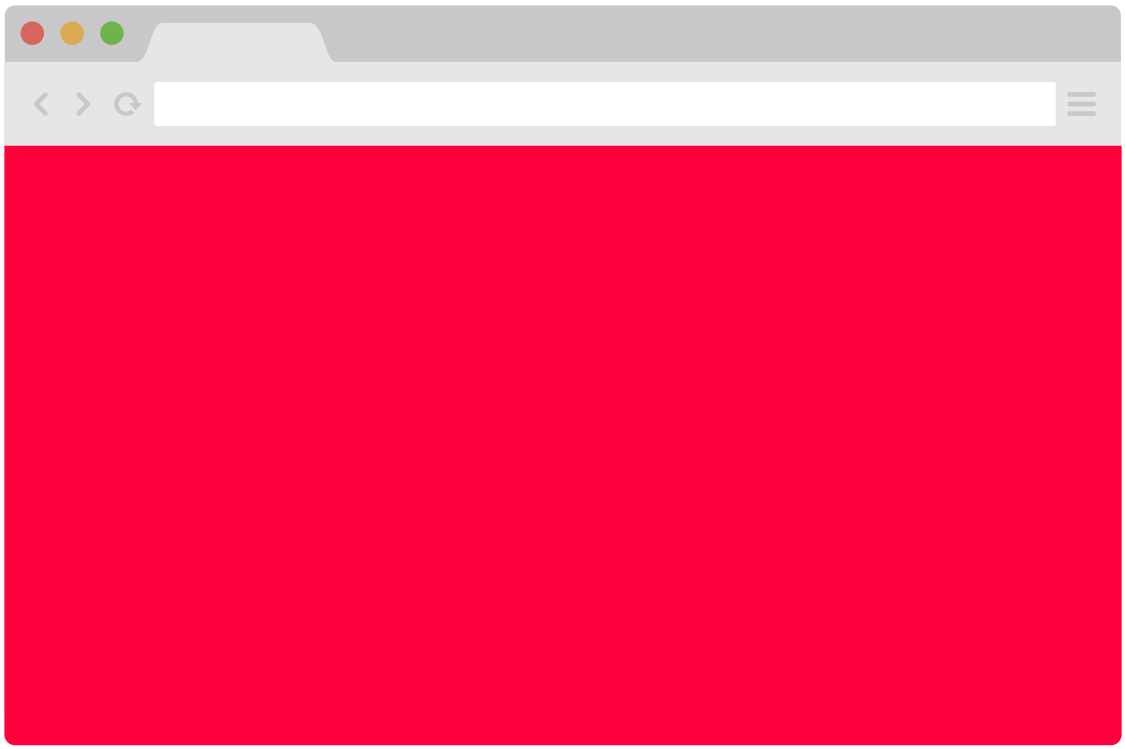


Fig. A static red page

If you don't expect your application to be popular, you can deploy a single instance of it.

Users visiting the website are redirected to the single running instance.

If your app is popular and you have to scale it to two

instances, you need to find a way to distribute the traffic. In the past, you might have used a load balancer.

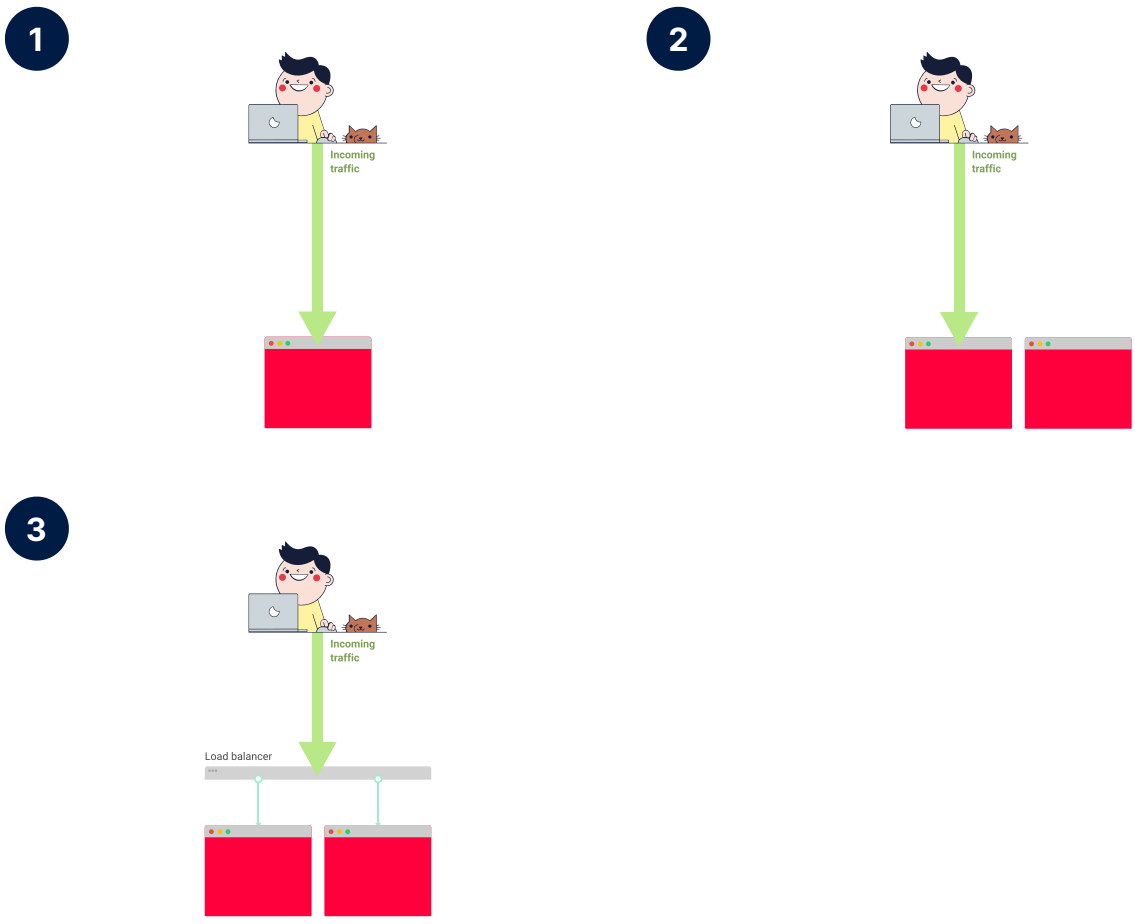



**Fig. 1** A user can visit the website by going directly to the app.

**Fig. 2** However, as soon as you scale the app to two instances, there's no easy way to distribute the traffic.

**Fig. 3** A load balancer is designed to evenly distribute the traffic to both instances.

You might find a similar challenge when instead of having an application deployed with a load balancer, you have two.

When you want to redirect traffic to both apps, you can use another load balancer located at the edge of your infrastructure.

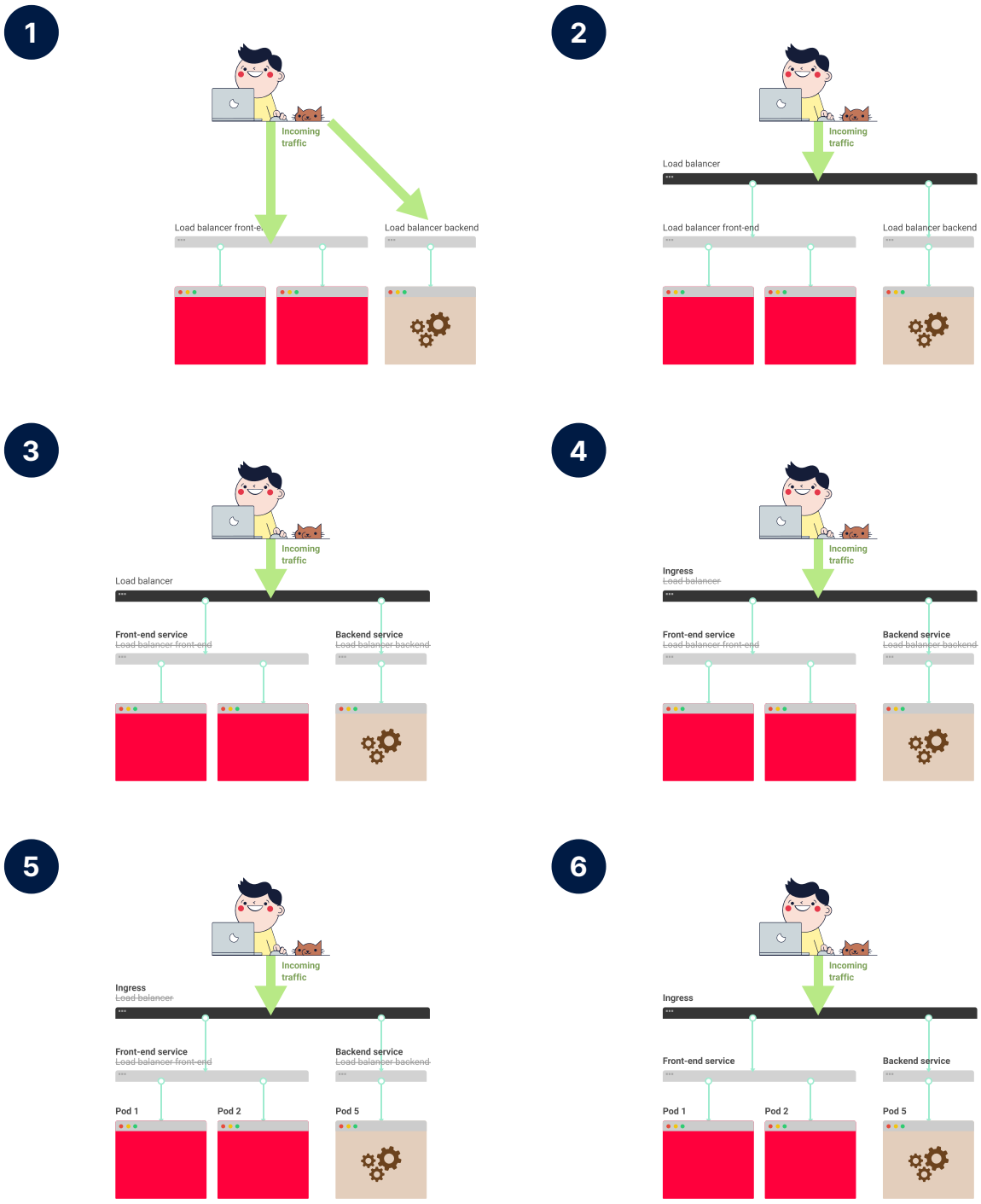



**Fig. 1** If you wish to expose both apps, you have to find a way to distribute the traffic, again.

**Fig. 2** Another load balancer could distribute the traffic to the internal load balancers and the apps.

**Fig. 3** In Kubernetes the internal load balancers are called *Services*.

**Fig. 4** The external load balancer is called an *Ingress*.

**Fig. 5** The instances receiving traffic are not called applications or containers. Instead they're called Pods.

**Fig. 6** Services, Ingress and Pods are fundamental concepts in Kubernetes. You will take the time to explore all of them in more details during this course.

As you can probably already imagine, each Pod has a dynamic IP address associated with it.

**Services are merely internal load balancers.**

They're designed to solve everyday challenges such as:

- **you don't need to keep track of how many Pods are deployed** — you request the Service, not an individual Pod.
- and even if a Pod is removed or added with a new IP address, **you don't need to update any routing table**

You can quickly scale your application to five Pods, and it will keep responding to the traffic with no downtime.

**The Ingress is like a router.**

You can set rules and have the traffic forwarded based on

paths, domain names, etc.

There's a fourth Kubernetes object that is fundamental to deploying Pods.

## Watching over Pods

*Pods are rarely deployed manually.*

When an application has a memory leak and is terminated, you'd expect the Pod to be respawned.

Most of the time, when you define a Pod, you should define it as part of a Deployment.

**A Deployment instructs Kubernetes to look after your Pods and make sure that they're healthy.**

When a Pod is lost, it's replaced by a new one.




**Fig. 1** In the picture, there is one Ingress, one Service and five Pods. There's another object that is not visible: the Deployment.

**Fig. 2** A Deployment is a recipe to create more Pods. You define what the Pod should look like and how many instances you wish to have.

**Fig. 3** The Deployment is also informing Kubernetes that five replicas should be alive at all times.

**Fig. 4** If for some reason one of the Pod is deleted, Kubernetes creates a new one.

**Fig. 5** If for some reason one of the Pod is deleted, Kubernetes creates a new one.

**Fig. 6** If for some reason one of the Pod is deleted, Kubernetes creates a new one.

A Deployment is a convenient way to deploy Pods.
You can configure a Deployment to run single or multiple replicas.
*How do you create Deployments, Services and Ingresses?*

Chapter 4

# Creating resources in Kubernetes

When you want to create a Deployment, Service or Ingress, you describe what the end state of the resources should be.

So instead of telling Kubernetes to create a Deployment with two replicas, you submit a request with all the details.

Kubernetes reads your wishlist, and it will try its best to create Pods, Services, etc.

People refer to this paradigm as *declarative* because you declare what the end state should be.

*But how do you declare those resources?*

Using a configuration language called YAML.

YAML, which stands for Yet Another Markup Language, is a human-readable text-based format for specifying configuration-type information.

YAML is a superset of JSON, which means that any valid JSON file is also a valid YAML file.

*Why not using JSON then?*

JSON is a repetitive format, and all values are wrapped into quotes `"`.

A format such as YAML is more concise and more comfortable to type.

Let's have a look at an example.

Pay attention to the following JSON example:

example.json

```
{
```

```
  "contributors": [
    {
      "name": "John Doe",
      "email": "johndoe@example.com"
    },
    {
      "name": "Ivy Lane",
      "url": "https://example.com/ivylane"
    }
  ],
  "dependencies": {
    "dep1": "^1.0.0",
    "dep2": "3.40",
    "dep3": "6.71"
  }
}
```

Now have a look at the same definition in YAML:

example.yaml

```
contributors:
- name: John Doe
  email: johndoe@example.com
- name: Ivy Lane
  url: https://example.com/ivylane
dependencies:
  dep1: "^1.0.0"
  dep2: '3.40'
  dep3: '6.71'
```

The format is less tedious and requires less indentation and quotes `"`.
Using YAML for Kubernetes objects gives you several advantages, including:

- **Convenience** — you don't have to add all of your

- parameters to the command line
- **Maintenance** — YAML files can be added to source control so that you can track changes
- **Flexibility** — you can create complex structures

There are two types of structures that you should familiarise in YAML:

- Lists
- Maps

Those aren't the only two types, but those are the ones that usually create confusion.

> If you wish to learn in detail all the different types, you should have a look at this concise documentation.

## YAML maps

Let's start by looking at YAML maps.
Maps let you associate key-value pairs.
For example, you might have a config file that starts like this:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
```

You can think of it in terms of its JSON equivalent:

pod.json

```json
{
  "apiVersion": "v1",
  "kind": "Pod"
}
```

You can also specify more complicated structures by creating a key that maps to another map as in:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
  labels:
    app: app
```

In this case, you have a key, `metadata`, that has as its value a map with two more keys, `name` and `labels`.
The `labels` key itself has a map as its value.
**You can nest these as far as you want to.**
The YAML processor knows how all of these pieces relate to each other because you indented the lines.

It's common practice to use two spaces for readability, but the number of spaces doesn't matter — as long as it's at least 1, and as long as you're **consistent**.
The following YAML is equivalent to the previous but uses 4 spaces for indentation:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
    name: example-pod
    labels:
        app: app
```

If you decide to translate the YAML into JSON, both definitions are equivalent to:

pod.json

```json
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "name": "example-pod",
    "labels": {
      "app": "app"
    }
  }
}
```

*What if you make a mistake?*
*How do you know if the YAML file is valid?*
You could use an online YAML linter or a command line

tool to verify your YAML files.
Let's have a look at lists next.

## YAML lists

YAML lists are a sequence of objects.
For example:

args.yaml

```yaml
args:
- sleep
- "1000"
```

You can have virtually any number of items in a list, which is defined as items that start with a dash `-` indented from the parent.
The YAML file translates to JSON as:

args.json

```json
{
  "args": ["sleep", "1000"]
}
```

Please note that members of the list can also be maps:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: example-pod
  labels:
    app: web
spec:
  containers:
  - name: app
    image: nginx
    ports:
    - containerPort: 88
```

So as you can see here, we have a list of containers "objects", each of which consists of:

- a `name`
- an `image`, and
- a list of `ports`

Each list item under ports is itself a map that lists the `containerPort` and its value.

For completeness, let's quickly look at the JSON equivalent.

Copy the content from above and try to convert it using this online editor.

As you can see, the definition is more complicated, but YAML is making it more readable.

# Getting your hands dirty

Now that you're familiar with the basic Kubernetes concepts, you should try and deploy an application for real.

Chapter 5

# Prerequisites

You are about to deploy an application to Kubernetes.
But first, you should provision a cluster.
The easiest option is to use `minikube` — a local cluster.
To start this module, you should have installed:

- `minikube`
- `kubectl`
- VirtualBox (Linux only)

You can follow the instruction guide for:

- Windows 10
- macOS
- Ubuntu

# Windows 10

You can download minikube with:

```
PowerShell                                    — ◻ X
PS> choco install minikube -y
```

You can test your minikube installation with:

```
PowerShell                                    — ◻ X
```

```
PS> minikube start
👍   Starting control plane node minikube in cluster minikube
🐋   Preparing Kubernetes v1.23.0 ...
🔎   Verifying Kubernetes components...
🌟   Enabled addons: default-storageclass, storage-provisioner
🏄   Done! kubectl is now configured to use "minikube" by default
```

PowerShell — □ X

```
PS> minikube delete
🔥   Deleting "minikube" ...
💀   Removed all traces of the "minikube" cluster.
```

And create a new one with:

PowerShell — □ X

```
PS> minikube start --v=7 --alsologtostderr
```

The `--v=7` flag increases the logging level, and you should be able to spot the error in the terminal output.

The extra verbose logging should help you get to the issue. In a particular case, minikube used to fail with:

> *"E0427 09:19:10.114873 10012 start.go:159] Error starting host: Error starting stopped host: exit status 1."*

Not very helpful. After enabling verbose logging, the issue was more obvious:

```
PowerShell
Hyper-V\Start-VM minikube
~~~~~~~~~~~~~~~~~~~~~~~~~
CategoryInfo          : FromStdErr: (:) [Start-VM], VirtualizationException
FullyQualifiedErrorId : OutOfMemory,Microsoft.HyperV.PowerShell.Commands.StartVM
```

I was running out of memory!

It's time to test if your installation has been successful. Before you do so, in the command prompt type:

```
PowerShell
PS> kubectl get nodes
NAME       STATUS   ROLES    AGE
minikube   Ready    master   88s
```

You should be able to see a single node in Kubernetes.

# macOS

You can download minikube with:

```
bash
$ brew install minikube
==> Downloading …
######################################################################## 100.0%
==> Installing dependencies for minikube: kubernetes-cli
==> Installing minikube dependency: kubernetes-cli
```

```
==> Pouring kubernetes-cli.tar.gz
==> Summary
🍺  /usr/local/Cellar/kubernetes-cli: 246 files, 46.1MB
==> Installing minikube
==> Pouring minikube.tar.gz
==> Summary
🍺  /usr/local/Cellar/minikube: 8 files, 62.4MB
```

> Please note how `kubectl` is automatically installed when you install `minikube`.

You can test your minikube installation with:

> Please note that running minikube as `root` is discouraged. Please run `minikube` as a standard user and not `sudo`.

bash

```
$ minikube start
👍  Starting control plane node minikube in cluster minikube
🏄  Preparing Kubernetes v1.23.0 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

If for any reason minikube fails to start up, you can debug it with:

bash

```
$ minikube delete
🔥  Deleting "minikube" ...
💀  Removed all traces of the "minikube" cluster.

$ minikube start --v=7 --alsologtostderr
```

The extra verbose logging should help you get to the issue.
It's time to test if your installation is successful.
Before you do so, in the command prompt type:

bash

```
$ kubectl get nodes
NAME       STATUS   ROLES    AGE
minikube   Ready    master   88s
```

*One last thing.*
Since kubectl makes requests to the Kubernetes API, it's worth checking that you are not using an older version of kubectl.

bash

```
$ kubectl version
Client Version: version.Info{Major:"1", Minor:"19", GitVersion:"v1.18.3"}
Server Version: version.Info{Major:"1", Minor:"18", GitVersion:"v1.18.3"}
```

**The Kubernetes API supports up to two older versions for backwards compatibility.**
So you could have the client (kubectl) on version 1.16 and the server on 1.18.

But you cannot use kubectl on version 1.15 with a cluster that runs on 1.19.

# Ubuntu

Before you install minikube, you should install VirtualBox.

## Installing VirtualBox

You can install VirtualBox with:

bash

```
$ sudo apt-get install -qqy --no-upgrade \
  linux-headers-`uname -r` \
  virtualbox
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
# truncated output
Do you want to continue? [Y/n] Y
done.
```

You can start VirtualBox with:

bash

```
$ virtualbox
```

## Installing kubectl

Add the Kubernetes repository to apt with:

bash

```
$ curl -s \
  https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
OK

$ sudo tee /etc/apt/sources.list.d/kubernetes.list > /dev/null <<EOF
deb http://apt.kubernetes.io/ kubernetes-xenial main
EOF
```

You can install `kubectl` with:

bash

```
$ sudo apt-get -qq update && sudo apt-get -qqy install kubectl
$ Unpacking kubectl ...
Setting up kubectl ...
```

## minikube

You can download minikube with:

bash

```
$ curl -Lo minikube \
  https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Current
                                 Dload  Upload   Total   Spent    Speed
100 55.2M  100 55.2M    0     0  11.4M      0  0:00:04  0:00:04 15.3M
```

You can install it with:

```bash
$ chmod +x minikube && sudo mv minikube /usr/local/bin/
```

You can test your minikube installation with:

> Please note that running minikube as `root` is discouraged. Please run minikube as a standard user and not `sudo`.

```bash
$ minikube start --vm
👍  Starting control plane node minikube in cluster minikube
🐋  Preparing Kubernetes v1.23.0 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

> VirtualBox isn't the default driver for Ubuntu, but it's the most stable.

If for any reason minikube fails to start up, you can debug it with:

```
PowerShell
PS> minikube delete
🔥  Deleting "minikube" ...
💀  Removed all traces of the "minikube" cluster.
```

And create a new one with:

```
PowerShell                                                    — □ X

PS> minikube start --v=7 --alsologtostderr
```

The `--v=7` flag increases the logging level, and you should be able to spot the error in the terminal output.

It's time to test if your installation was successful. In the command prompt type:

```
bash

$ kubectl get nodes
NAME       STATUS   ROLES    AGE
minikube   Ready    master   88s
```

Great!

Chapter 6

# First deployment

In this chapter, you will deploy your container in a Kubernetes cluster.

If you followed the instructions in the prerequisites chapter, you should already have `minikube` and `kubectl` installed.

## Starting the local cluster

You can launch minikube with:

INSTRUCTIONS FOR MACOS

bash

```
$ minikube start --driver=virtualbox
👍  Starting control plane node minikube in cluster minikube
🏄  Preparing Kubernetes v1.23.0 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

INSTRUCTIONS FOR LINUX

bash

```
$ minikube start --driver=virtualbox
👍  Starting control plane node minikube in cluster minikube
🏄  Preparing Kubernetes v1.23.0 ...
🔎  Verifying Kubernetes components...
🌟  Enabled addons: default-storageclass, storage-provisioner
🏄  Done! kubectl is now configured to use "minikube" by default
```

INSTRUCTIONS FOR WINDOWS

PowerShell — ◻ X

```
PS> minikube start --driver=hyperv
👍   Starting control plane node minikube in cluster minikube
🐳   Preparing Kubernetes v1.23.0 ...
🔎   Verifying Kubernetes components...
🌟   Enabled addons: default-storageclass, storage-provisioner
🏄   Done! kubectl is now configured to use "minikube" by default
```

As soon as minikube starts, you can check that your cluster is ready with:

bash

```
$ kubectl cluster-info
Kubernetes master is running at https://192.168.64.20:8443
KubeDNS is running at <long url>
```

**You have a fully-functioning Kubernetes cluster on your machine now.**

Before moving forward, it's worth learning a few tricks to troubleshoot your setup.

When things go wrong with your minikube, and you think it might be broken, you can delete minikube with:

bash

```
$ minikube delete
🔥   Deleting "minikube" ...
💀   Removed all traces of the "minikube" cluster.
```

You can start a new minikube cluster with the same `minikube start` command. Since kubectl makes requests

to the Kubernetes API, it's worth checking that you are not using an older version of kubectl.

bash

```
$ kubectl version
Client Version: version.Info{Major:"1", Minor:"19", GitVersion:"v1.18.3"}
Server Version: version.Info{Major:"1", Minor:"18", GitVersion:"v1.18.3"}
```

**The Kubernetes API supports up to two older versions for backwards compatibility.**

So you could have the client (kubectl) on version 1.16 and the server on 1.18.

But you cannot use kubectl on version 1.15 with a cluster that runs on 1.19.

## Your first deployment

**Podinfo is a web application that showcases best practices of running apps in Kubernetes.**

As soon as you launch it, this is what it looks like:

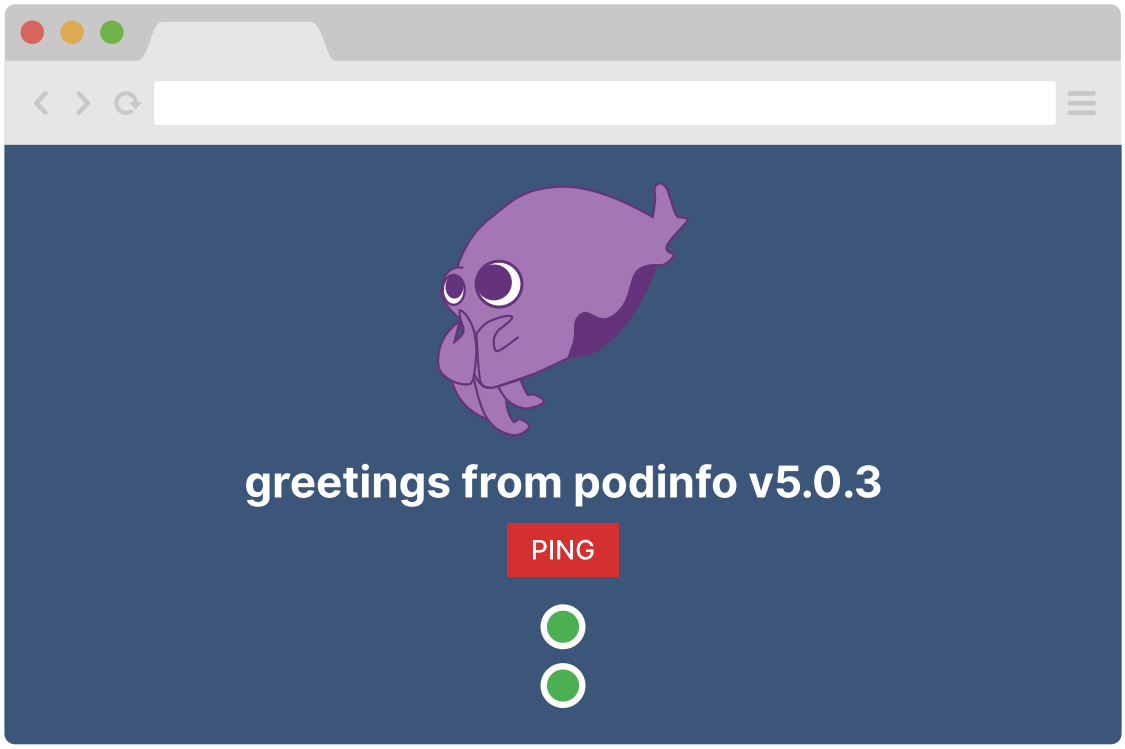



**Fig.** Podinfo is a tiny web application made with Go that showcases best practices of running microservices in Kubernetes.

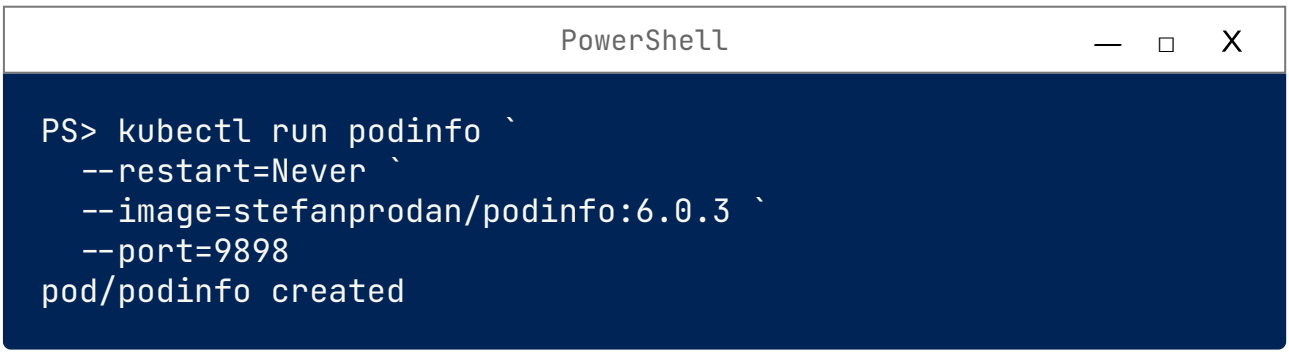
The container is available publicly on Docker Hub.
You can deploy PodInfo on Kubernetes with:

INSTRUCTIONS FOR BASH

bash

```
$ kubectl run podinfo \
  --restart=Never \
  --image=stefanprodan/podinfo:6.0.3 \
  --port=9898
pod/podinfo created
```

INSTRUCTIONS FOR POWERSHELL




Let's review the command:

- `kubectl run podinfo` deploys an app in the cluster and gives it the name `podinfo`.
- `--restart=Never` is used not to restart the app when it crashes.
- `--image=stefanprodan/podinfo:6.0.3` and `--port=9898` are the name of the image and the port exposed on the container.

> Please note that 9898 is the port exposed in the container. If you make a mistake, you shouldn't increment the port; you can still use port 9898. If you make a mistake, you can execute `kubectl delete pod app` and start again.

If the command was successful, there is a good chance that Kubernetes deployed your application.
You should check that the cluster has a new Pod with:

```bash
$ kubectl get pods
NAME              READY   STATUS    RESTARTS
podinfo           1/1     Running   0
```

Great!
*But how can you reach that Pod and visit it in a browser?*
You should expose it.
You can expose the `podinfo` Pod with a Service:

```bash
$ kubectl expose pod podinfo --port=9898 --type=NodePort
service/podinfo exposed
```

Let's review the command:

- `kubectl expose pod podinfo` exposed the Pod named `podinfo`.
- `--port=9898` is the port exposed by the container.
- `--type=NodePort` is the type of Service. Kubernetes has four different kinds of Services. NodePort is the Service that exposes apps to external traffic.

You should check if Kubernetes created the Service with:

```bash
$ kubectl get services
NAME         TYPE        CLUSTER-IP       PORT(S)
podinfo      NodePort    10.111.188.196   9898:32036/TCP
```

To obtain the URL to visit your application, you should run:

```bash
$ minikube service --url podinfo
http://192.168.64.18:32036
```

If you visit that URL in the browser (yours might be slightly different), you should see the app in full.

*What did* `kubectl run` *and* `kubectl expose` *do?*

## Inspecting YAML resources

Kubernetes created a few resources when you typed `kubectl run`, and `kubectl expose`.

When you ran `kubectl run podinfo` it created a Pod object.

You can verify that with:

```bash
$ kubectl get pods
NAME              READY   STATUS    RESTARTS
podinfo           1/1     Running   0
```

However:

1. **Kubernetes has a declarative interface** — you describe what you want, not how you get there. In this instance, you are telling Kubernetes how to run your pod.
2. **All workloads are defined in YAML.** But you haven't written any YAML so far, and deployment worked anyway.

`kubectl run` and `kubectl expose` are two shortcuts that are useful for creating resources quickly.

However, they are rarely used in a production environment.

When you type `kubectl run`, a YAML definition is automatically created, populated and submitted to the cluster.

You can inspect the YAML submitted to the cluster with:

```bash
$ kubectl get pod podinfo --output yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: podinfo
```

```
  name: podinfo
  namespace: default
  # truncated output
spec:
  containers:
  - image: stefanprodan/podinfo:6.0.3
    imagePullPolicy: Always
    name: podinfo
    ports:
    - containerPort: 9898
      protocol: TCP
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    volumeMounts:
    - mountPath: /var/run/secrets/kubernetes.io/serviceaccount
      name: default-token-lvsst
      readOnly: true
  # truncated output
status:
  # truncated output
```

The output is overwhelming!

*How can I short command like* `kubectl run` *generate over 150 lines of YAML code?*

The response includes:

- The status of the resource in `status`.
- The schema of the resource in `metadata.managedFields`.
- The propriety of the pod in `spec`.

Most of the fields are set to their default value, and some of them are automatically included by your cluster (and may vary).

*With time, you will learn all of those, but for now, let's focus on the basics.*

The long output can be shortened to this:

pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: podinfo
spec:
  containers:
    - name: podinfo
      image: stefanprodan/podinfo:6.0.3
      ports:
        - containerPort: 9898
```

The name of the Pod, the image and the container port are the same values that you provided to `kubectl run`.

The command generated the above YAML and submitted to the cluster for you.

As you can imagine, `kubectl expose` works in a similar way.

As you type the command, kubectl generates a Service.

Let's explore the YAML stored in the cluster with:

bash

```
$ kubectl get services podinfo --output yaml
apiVersion: v1
kind: Service
metadata:
  labels:
```

```
    run: podinfo
  name: podinfo
  namespace: default
  # truncated output
spec:
  clusterIP: 10.111.188.196
  externalTrafficPolicy: Cluster
  ports:
  - nodePort: 32036
    port: 9898
    protocol: TCP
    targetPort: 9898
  selector:
    run: podinfo
  sessionAffinity: None
  type: NodePort
status:
  loadBalancer: {}
```

The output is verbose, and there are a few fields that are automatically set to their defaults.

The above YAML shortened could be rewritten as:

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: podinfo
spec:
  selector:
    run: podinfo
  ports:
    - port: 9898
      targetPort: 9898
      nodePort: 32036
```

The YAML definition includes:

- The name of the resource in `metadata.name`.
- The port expose by the Service as `spec.ports[0].port`.
- The port used to forward traffic to the Pods `spec.ports[0].targetPort`.
- The port that is exposed to public traffic `spec.ports[0].nodePort`.

The following diagram is a visual representation of the Service's ports.



**Fig. 1** The Service above exposes three ports: `port`, `targetPort` and `nodePort`.

**Fig. 2** The `targetPort` is used to forward traffic to the Pods and should always match the `containerPort` in the Pod resource.

**Fig. 3** The `nodePort` is a convenient way to bypass the Ingress and expose the Service directly from the cluster.

**Fig. 4** The `port` is used by any service in the cluster to communicate with the Pods. It's also used by the Ingress to locate the Service.

Object definitions for Pods and Services are written in YAML.

But YAML isn't the only option.

You can retrieve the same Pod in JSON format with:

bash

```
$ kubectl get pod podinfo --output json
{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "creationTimestamp": "2020-10-23T01:51:52Z",
    "labels": {
      "run": "podinfo"
    },
# truncated output
```

And you can do the same with the Service too:

bash

```
$ kubectl get service podinfo --output json
{
  "apiVersion": "v1",
  "kind": "Service",
  "metadata": {
    "creationTimestamp": "2020-10-23T01:55:20Z",
    "labels": {
      "run": "podinfo"
    },
# truncated output
```

*How do you remember all the available properties for objects such as Services and Pods?*

There are two useful strategies to explore all available configuration options.

The first is the official Kubernetes API.

The second is the `kubectl explain` command.

You can use the command to query the same description that you can read from the API like this:

bash

```
$ kubectl explain pod.spec.containers
KIND:     Pod
VERSION:  v1

RESOURCE: containers <[]Object>

DESCRIPTION:
     List of containers belonging to the pod. Containers cannot currently be
     added or removed. There must be at least one container in a Pod. Cannot be
     updated.

     A single application container that you want to run within a pod.
```

**Kubectl is probably one of your most-used tools in Kubernetes.**

Whenever you spend a lot of time working with a specific tool, it is worth to get to know it very well and learn how to use it efficiently.

At the back of this book, you will find some bonus material on how to get more proficient with it.

## Exposing the application

Exposing applications to the public using Services is not common.

Services are similar to internal load balancers and can't route traffic depending on the path, for example.

And that's a frequent requirement.

Imagine having two apps:

- The first should handle the traffic to `/store`.
- The second should serve requests for `/api`.

*How do you split the traffic and route it to the appropriate Pod?*

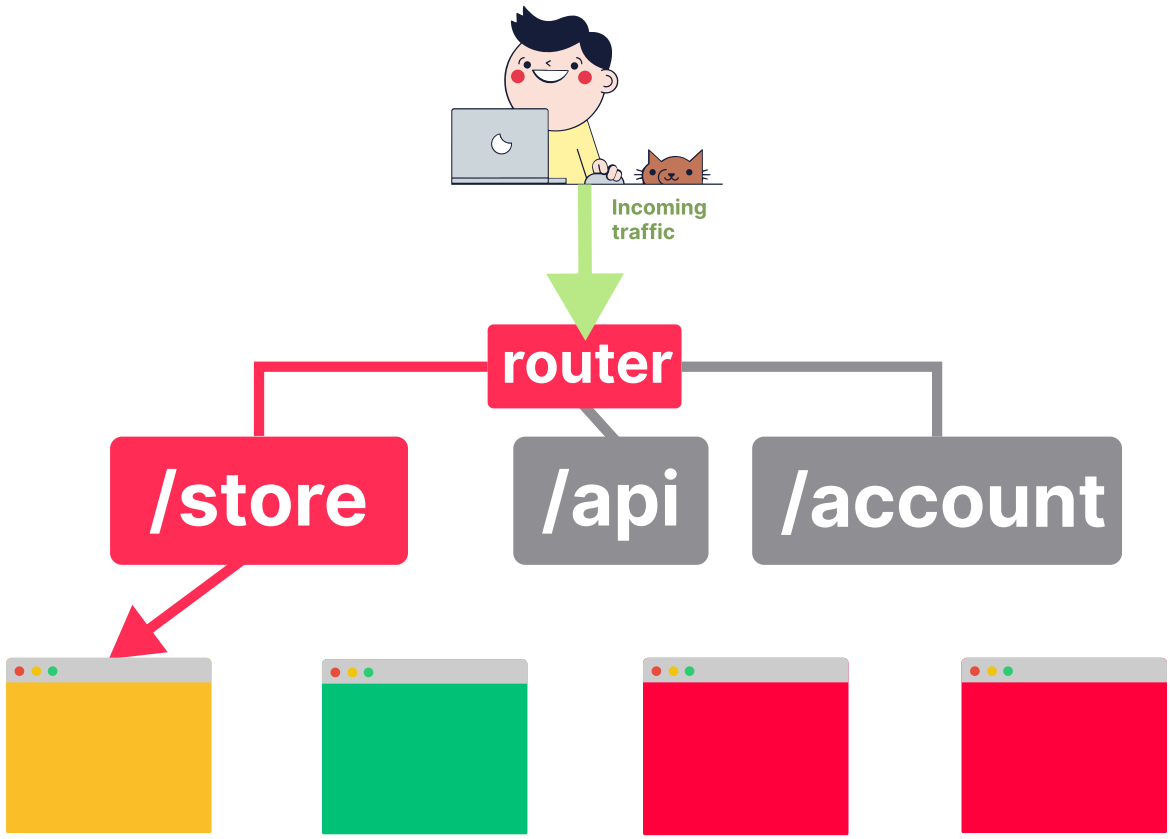



Fig. Routing the traffic based on paths

You might have used a tool such as Nginx to route the traffic to several apps.

You define rules, such as the domain or the path, and the backend that should receive the traffic.

Nginx inspects every incoming request against the rules and forwards it to the right backend.

**Routing traffic in such manner is popular even in**

**Kubernetes.**

Hence there's a specific object called Ingress where you can define routing rules.

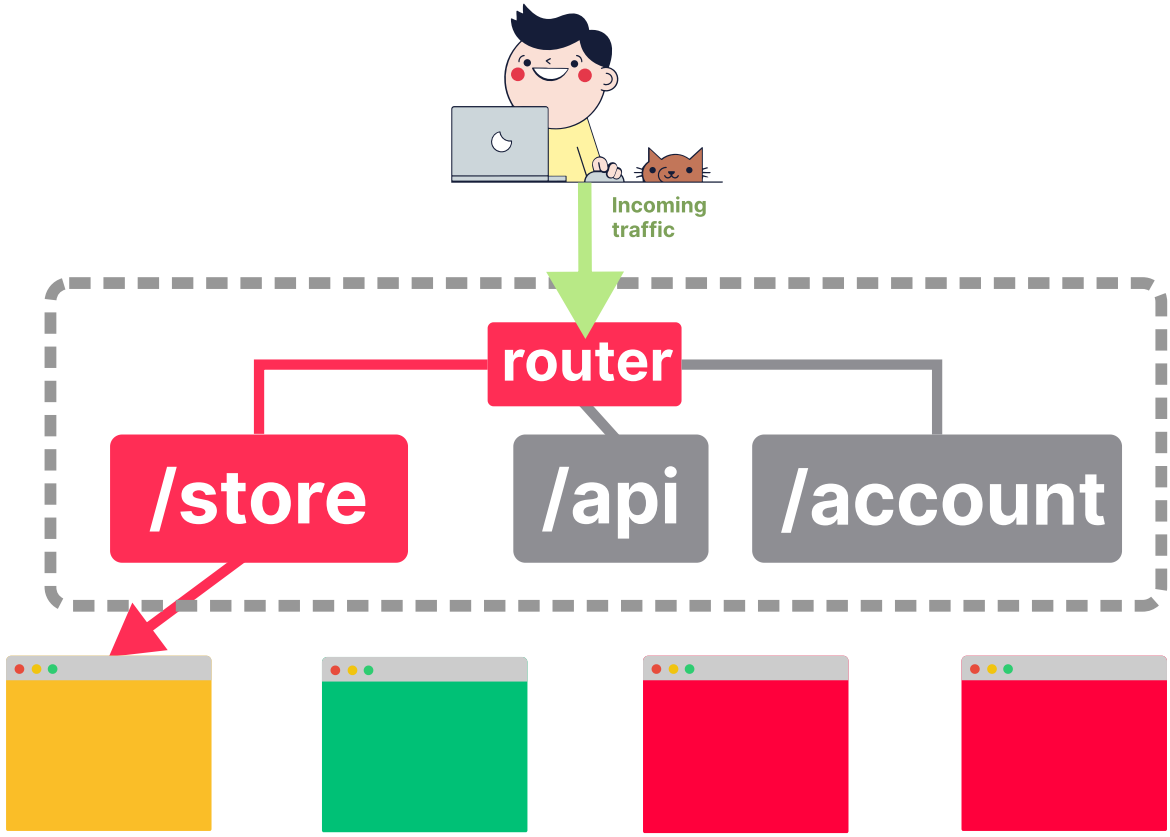



**Fig.** Routing the traffic based on paths in Kubernetes with the Ingress

Instead of using a Service, you should create an Ingress to expose your application to the public internet.

You explored `kubectl run` and `kubectl expose` that

created YAML resources on your behalf.
This time, you will create the Ingress resource by hand.

> Please note that creating resource manually in YAML is considered best practice. Commands such as `kubectl run` and `kubectl expose` are not popular in the Kubernetes community, but they are an excellent start to familiarise with Kubernetes.

Create an `ingress.yaml` file with the following content:

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: podinfo
spec:
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: podinfo
            port:
              number: 9898
```

*Let's break down that YAML definition.*

Every time the incoming request matches the path `/`, the traffic is forwarded to the Pods targeted by the `podinfo` Service on port 9898.

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: podinfo
spec:
  rules:
    - http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: podinfo
                port:
                  number: 9898
```

Before you submit the resource to the cluster, you should enable the Ingress controller in `minikube`:

bash

```
$ minikube addons enable ingress
🔎  Verifying ingress addon...
🌟  The 'ingress' addon is enabled
```

> Please note that the Ingress could take some time to download. You can track the progress with `kubectl get pods --all-namespaces`.

You can submit the Ingress manifest with:

```bash
$ kubectl apply --filename ingress.yaml
ingress.networking.k8s.io/podinfo created
```

> The command is usually shortened to `kubectl apply -f`.

You should verify that Kubernetes created the resource with:

```bash
$ kubectl get ingress podinfo
NAME      CLASS    HOSTS   ADDRESS         PORTS
podinfo   <none>   *       192.168.64.18   80
```

If you wish to inspect the resource stored in the cluster, you can do so with:

```bash
$ kubectl get ingress podinfo --output yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
  # truncated output
```

*Does it work?*

Let's find out.

First, retrieve the IP address of minikube with:

```bash
$ minikube ip
192.168.64.18
```

Visit the `http://[replace with minikube ip]/` in your browser.

**Congratulations!**

It's the same app, but this time it's served by the Ingress.

Notice how the traffic goes to port 80 on the Minikube's IP address and then to the Pod.

If you recall, the Service used a random port in the range of 30000.

## Recap

A quick recap of what you've done so far:

1. You created a single Pod.
2. You exposed the Pod using a Service.
3. You exposed the application to the public using an Ingress.

The deployment works, but you could provide better reliability and scalability with a Deployment object.

Instead of deploying the Pod manually, you could wrap the definition in a Deployment and have Kubernetes creating it on your behalf.

As a benefit, Kubernetes monitors and restarts the Pod.

*You can also scale the number of instances by changing the value in the deployment.*

In the next chapter, you will explore the Deployment resource in Kubernetes.

Chapter 7

# Self-healing and scaling

At this point you should have a local Kubernetes cluster with:

- A single Pod running
- A Service that routes traffic to a Pod
- An Ingress that exposes the Pod to external sources

Having a single Pod is usually not enough.
*What happens when the Pod is accidentally deleted?*
It's time to find out.

## Practicing chaos engineering

You can delete the Pod with:

```bash
$ kubectl delete pod podinfo
pod "podinfo" deleted
```

If you visit the app with `http://[replace with minikube ip]/`, does it still work?
*"503 Service Temporarily Unavailable".*
**But why?**
You deployed a single Pod in isolation.

**There's no process looking after and respawning it when it's deleted.**

As you can imagine, a single Pod that isn't recreated is of limited use.

*It'd be better if there could be a mechanism to watch Pods and restart them when they are deleted.*

Kubernetes has an abstraction designed to solve that specific challenge: the Deployment object.

Here's an example for a Deployment definition:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: first-deployment
spec:
  selector:
    matchLabels:
      run: podinfo
  template:
    metadata:
      labels:
        run: podinfo
    spec:
      containers:
      - name: podinfo
        image: stefanprodan/podinfo:6.0.3
        ports:
        - containerPort: 9898
```

Notice that, since the Deployment watches over Pod, it also embeds the definition of a Pod.

If you pay close attention, `spec.template` is the definition

of a Pod.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: first-deployment
spec:
  selector:
    matchLabels:
      run: podinfo
  template:
    metadata:
      labels:
        run: podinfo
    spec:
      containers:
      - name: podinfo
        image: stefanprodan/podinfo:6.0.3
        ports:
        - containerPort: 9898
```

The `spec.selector` field is used to select which Pods the Deployment should look after.
Kubernetes uses labels and selectors to link the Deployments and the Pods.

> Please note how the label is a key-value pair.

In this case, the Deployment monitors all the pods with a `run: podinfo` label.
The label is repeated in the Pod definition.

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: first-deployment
spec:
  selector:
    matchLabels:
      run: podinfo
  template:
    metadata:
      labels:
        run: podinfo
    spec:
      containers:
      - name: podinfo
        image: stefanprodan/podinfo:6.0.3
        ports:
        - containerPort: 9898
```

You can copy the content of the above file and save it as `deployment.yaml`.

You can create the Deployment in Kubernetes with:

bash

```
$ kubectl apply -f deployment.yaml
deployment.apps/first-deployment created
```

The Deployment should create a Pod that has the same label as the one that was deleted earlier.

If you visit `http://[replace with minikube ip]/`, *can you see the app?*

Yes, it worked.

*Did the Deployment create a Pod?*
Let's find out:

bash

```
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS
first-deployment-w4wsp        1/1     Running   0
```

The Deployment created a single Pod.
*What happens when you delete it again?*

bash

```
$ kubectl delete pod [insert pod id]
```

*The Deployment immediately respawns another Pod.*
You can verify that is the case by visiting the URL in your browser or with:

bash

```
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS
first-deployment-uer2x        1/1     Running   0
```

However, you *still* have a single Pod running.
*How could you serve more traffic when the application becomes more popular?*

# Scaling the application

**You could ask Kubernetes to scale your Deployment and create more copies of your Pod.**

Open a new terminal and watch the number of Pods with:

```bash
$ kubectl get pods --watch
NAME                          READY   STATUS    RESTARTS
first-deployment-uer2x        1/1     Running   0
```

In the previous terminal scale your Deployment to five replicas with:

```bash
$ kubectl scale --replicas=5 deployment first-deployment
deployment.apps/first-deployment scaled
```

You should see four more Pods in the terminal running the command `kubectl get pods --watch`:

```bash
$ kubectl get pods --watch
```

```
NAME                      READY   STATUS    RESTARTS
first-deployment-uer2x    1/1     Running   0
first-deployment-smd6z    1/1     Running   0
first-deployment-smq9t    1/1     Running   0
first-deployment-w4wsp    1/1     Running   0
first-deployment-w7sv4    1/1     Running   0
```

Unfortunately, `kubectl scale` is one of those imperative commands that is not very popular in the Kubernetes community.

You can try to edit the YAML and submit it to the cluster.

Let's scale up your deployment to ten instances by editing the YAML configuration in place:

bash

```
$ kubectl edit deployment first-deployment
```

And change the `spec.replicas` field to 10.

You should look at the other terminal and notice that Kubernetes spawned more Pods as a result.

*So which command should you use,* `kubectl edit` *or* `kubectl scale`*?*

If you can, you should avoid both.

In Kubernetes, you will create resources, change configurations and delete resources with YAML.

**So it's generally recommended to modify the YAML file on the filesystem and resubmit it to the cluster.**

In this particular instance, the right thing to do is to edit the `deployment.yaml` as follows:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: first-deployment
spec:
  replicas: 10
  selector:
    matchLabels:
      run: podinfo
  template:
    metadata:
      labels:
        run: podinfo
    spec:
      containers:
      - name: podinfo
        image: stefanprodan/podinfo:6.0.3
        ports:
        - containerPort: 9898
```

You can resubmit the resource to the cluster with:

bash

```
$ kubectl apply -f deployment.yaml
```

*Why not using `kubectl scale`?*

You can keep using `kubectl scale`, but it's something else that you have to remember on top of all the other commands.

Editing YAML is universal, and it's using the familiar `kubectl apply -f` command.

*Why not using* `kubectl edit` *?*

`kubectl edit` modifies the resource directly in the cluster.

While it's technically similar to `kubectl apply -f` it also has a drawback.

You don't have a record of the change.

When you use `kubectl apply` you are forced to have the file locally.

**That file can be stored in version control and kept as a reference.**

There are several benefits to this pattern:

- You can share your work with other colleagues.
- You know who committed the change and when.
- You can always revert the change to a previous version.

Since this is also a best practice, you will stop using commands such as `kubectl run` and `kubectl expose` and start writing YAML files from now on.

Now that you have ten replicas, you should handle ten times more requests.

*But who is distributing those requests to the ten Pods?*

# Scaling the Service

**The Service is in charge of distributing requests to Pods.**

Services are convenient when you have two apps in the cluster that want to communicate.

Instead of one Pod directly talking to another Pod, you have them going through a middleman: the Service.

In turn, the Service forwards the request to one of the Pods.

The Service is convenient because if a Pod is deleted, the request is forwarded to another Pod.

You don't need to know the specific Pod that receives the request; the Service chooses it for you.

Also, it doesn't have to know how many Pods are behind that Service.

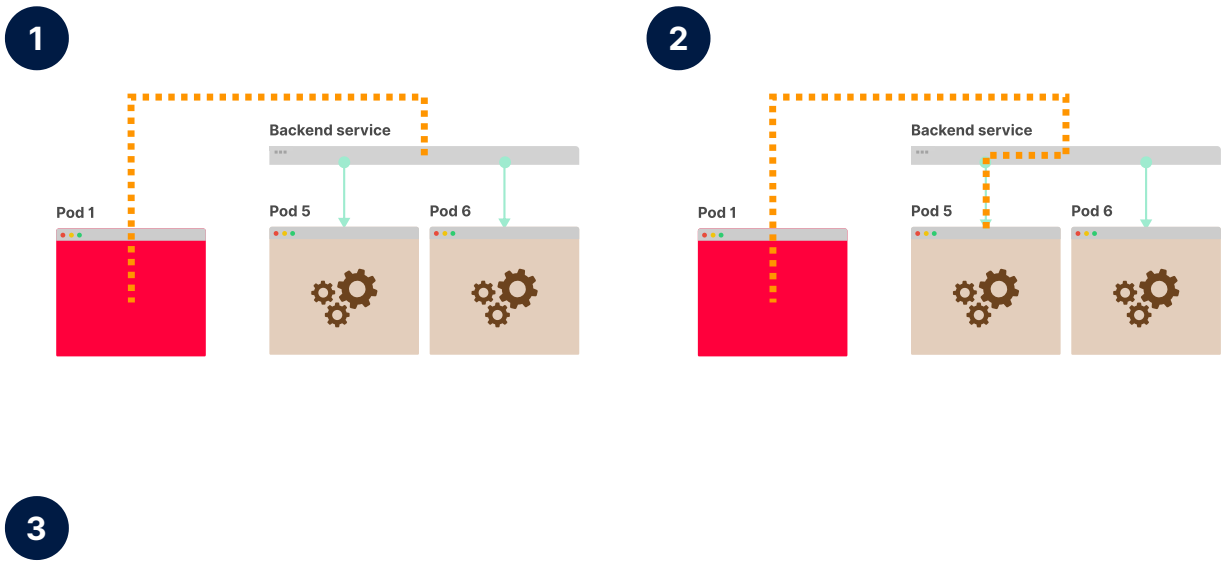


**Fig. 1** The red Pod issues a request to an internal (beige) component. Instead of choosing one of the Pods as the destination, the red Pod issues the request to the Service.

**Fig. 2** The Service forwards the traffic to one of the Pods.

**Fig. 3** Notice how the red Pod doesn't know how many replicas are hidden behind the Service.

## Recap

Your deployment in Kubernetes is rock solid.

When a Pod is deleted, it is automatically respawned.

You can change how many instances of your app are running with a line change in a YAML file.

You have a solid framework that can be copy-pasted and adapted to deployment.

You can reuse the same YAML file, and template the same values with different names.

In your Kubernetes journey, you will

- Not create Pods manually, but use Deployments instead.
- Often create pairs of Deployment and Service.
- Occasionally create Ingress resources to expose a few

- selected Services in the cluster.

Until now, the Deployment you created used the premade container image `stefanprodan/podinfo:6.0.3`.

*What if you wish to deploy your image?*

In the next chapter, you will code, package and deploy a simple application with what you've learned so far.

Chapter 8

# Deploying an app on Kubernetes end-to-end

Applications deployed in Kubernetes are usually packaged as containers.

*So let's explore the workflow to build and package applications to deploy them in Kubernetes.*

Since the focus is not on development, you will deploy the following static web page served by Nginx.




Fig. A static web page served by Nginx

# Creating the application

You should create a directory where you can save the files:

bash

```
$ mkdir -p first-k8s-app
$ cd first-k8s-app
```

Next, you should create an `index.html` file with the following content:

```
<html style="background-color:#FF003C">
  <head>
    <title>v1.0.0</title>
  </head>
  <body style="display:flex;
               align-items:center;
               justify-content:center;
               color:#FFFFFF;
               font-family:sans-serif;
               font-size:6rem;margin:0;
               letter-spacing:-0.1em">
    <h1>v1.0.0</h1>
  </body>
</html>
```

You can open the file in your browser of choice and see what it looks like.

*It's a boring HTML page!*

A static HTML file doesn't present the same challenges as a fully-fledged application written in Python, Node.js or Java.

It doesn't connect to a database or use CPUs to process data-intensive workloads.

However, it's enough to get an idea of what's necessary to upgrade your development workflow with Kubernetes.

You have the app, so it's time to package it as a container image.

## Containerising the application

First of all, you have to install the Docker Community Edition (CE).

You can follow the instructions in the official Docker documentation.

If you need help, you will find the instructions on how to install Docker at the back of this book.

You can verify that Docker is installed correctly with the following command:

bash

```
$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

**You're now ready to build Docker containers.**

Docker containers are built from `Dockerfile`s.

A `Dockerfile` is like a recipe — it defines what goes in a container.

A `Dockerfile` consists of a sequence of commands.

> You can find the full list of commands in the Dockerfile reference.

Here is a `Dockerfile` that packages your app into a container image:

Dockerfile

```
FROM nginx:1.19.5

EXPOSE 80
COPY index.html /usr/share/nginx/html

CMD ["nginx", "-g", "daemon off;"]
```

The file above reads as follows:

- `FROM nginx:1.19.5` is the base image. The image contains a minimal version of the Debian operating system with Nginx installed.
- `EXPOSE 80` is a comment and doesn't expose port 80. It's only a friendly reminder that the process exposes port 80.

- `COPY index.html /usr/share/nginx/html` copies the HTML file in the container.
- `CMD ["nginx", "-g", "daemon off;"]` launches the Nginx process without making it a background process.

You can build the image with:

bash

```
$ docker build -t first-k8s-app .
Sending build context to Docker daemon  3.072kB
Step 1/4 : FROM nginx:1.19.5
 ---> 5a8dfb2ca731
Step 2/4 : EXPOSE 80
 ---> Using cache
 ---> fcda535bf090
Step 3/4 : COPY index.html /usr/share/nginx/html
 ---> Using cache
 ---> 3581fd7eed08
Step 4/4 : CMD ["nginx", "-g", "daemon off;"]
 ---> Using cache
 ---> 253e0d2e6293
Successfully built 253e0d2e6293
Successfully tagged first-k8s-app:latest
```

Note the following about this command:

- `-t first-k8s-app` defines the name ("tag") of your container — in this case, your container is just called `first-k8s-app`.
- `.` is the location of the Dockerfile and application code — in this case, it's the current directory.

The command executes the steps outlined in the `Dockerfile`, one by one:

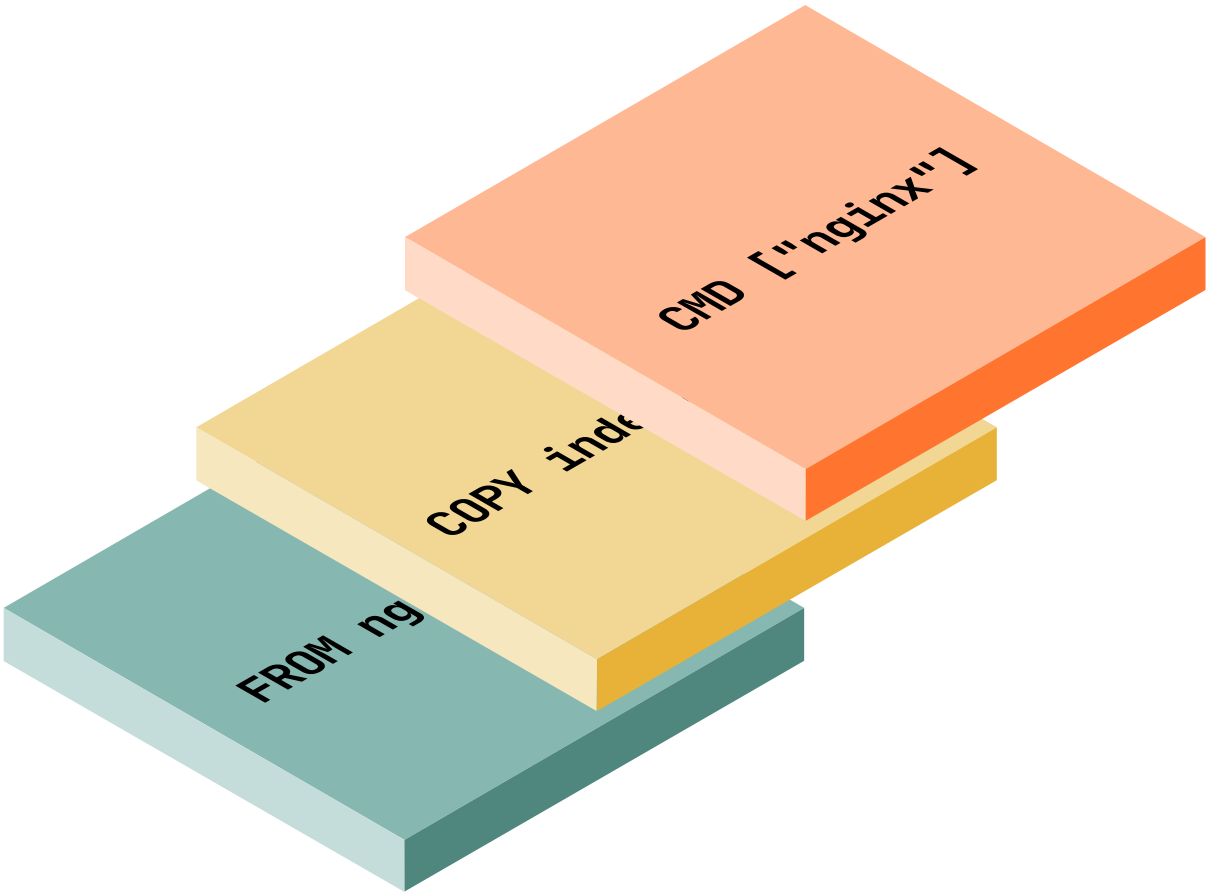



**Fig.** Docker creates a layer for every instruction in the Dockerfile.

### The output is a Docker image.

You can list all the images on your system with the following command:

bash

```
$ docker images
```

```
REPOSITORY      TAG       IMAGE ID       CREATED          SIZE
first-k8s-app   latest    dc2a8fd35e2e   30 seconds ago   165MB
nginx           1.17.9    d9bfca6c7741   2 weeks ago      150MB
```

You should see the `app` image that you built.

You should also see the `nginx:1.19.5` which is the base layer of your `app` image — it is just an ordinary image as well, and the `docker build` command downloaded it automatically from Docker Hub.

> Docker Hub is a container registry — a place to distribute and share container images.

*You packaged your app as a Docker image — let's run it as a container.*

## Running the container

You can run your app as follows:

bash

```
$ docker run -ti --rm -p 8080:80 first-k8s-app
```

Note the following about this command:

- `--rm` automatically cleans up the container and removes the file system when the container exits.
- `-p 8080:80` publishes port 80 of the container to port 8080 of your local machine. That means, if you now access port 8080 on your computer, the request is forwarded to port 80 of the app. You can use the forwarding to access the app from your local machine.
- `-ti` attached the output of the terminal to the current terminal session.

You can test that the container runs successfully with:

bash

```
$ curl localhost:8080
<html style="background-color:#FF003C">
  <head>
    <title>v1.0.0</title>
  </head>
  <body style="display:flex;
               align-items:center;
               justify-content:center;
               color:#FFFFFF;
               font-family:sans-serif;
               font-size:6rem;margin:0;
               letter-spacing:-0.1em">
    <h1>v1.0.0</h1>
  </body>
</html>
```

You can display all running containers with the following command:

```bash
$ docker ps
CONTAINER ID   IMAGE           COMMAND                  PORTS                    NAMES
41b50740a920   first-k8s-app   "nginx -g 'daemon of…"   0.0.0.0:8080->80/tcp   busy_allen
```

Great!

*It's time to test your application!*

You can visit http://localhost:8080 with your browser to inspect the containerised app.

When you're done experimenting, stop the container with Ctrl+C.

## Creating the image in Minikube

You built the current Docker image against your local Docker daemon.

But Minikube features its own Docker daemon.

When starting a Pod, minikube checks the local images first, before downloading them from the internet.

*How could you move the image between the two Docker daemons?*

In most cases, you should upload and download your image from the a registry such as Docker Hub, the GitHub container registry or a private registry such as Amazon ECR.

*However, that usually takes time.*

Imagine creating an image and uploading it, only to notice a small error in the Docker.

You'd have to start from scratch, recompile and redownload the container.

You can short your development cycle by leveraging Docker's client-server architecture.

You could connect to Minikube's Docker demon and build your images directly in the cluster.

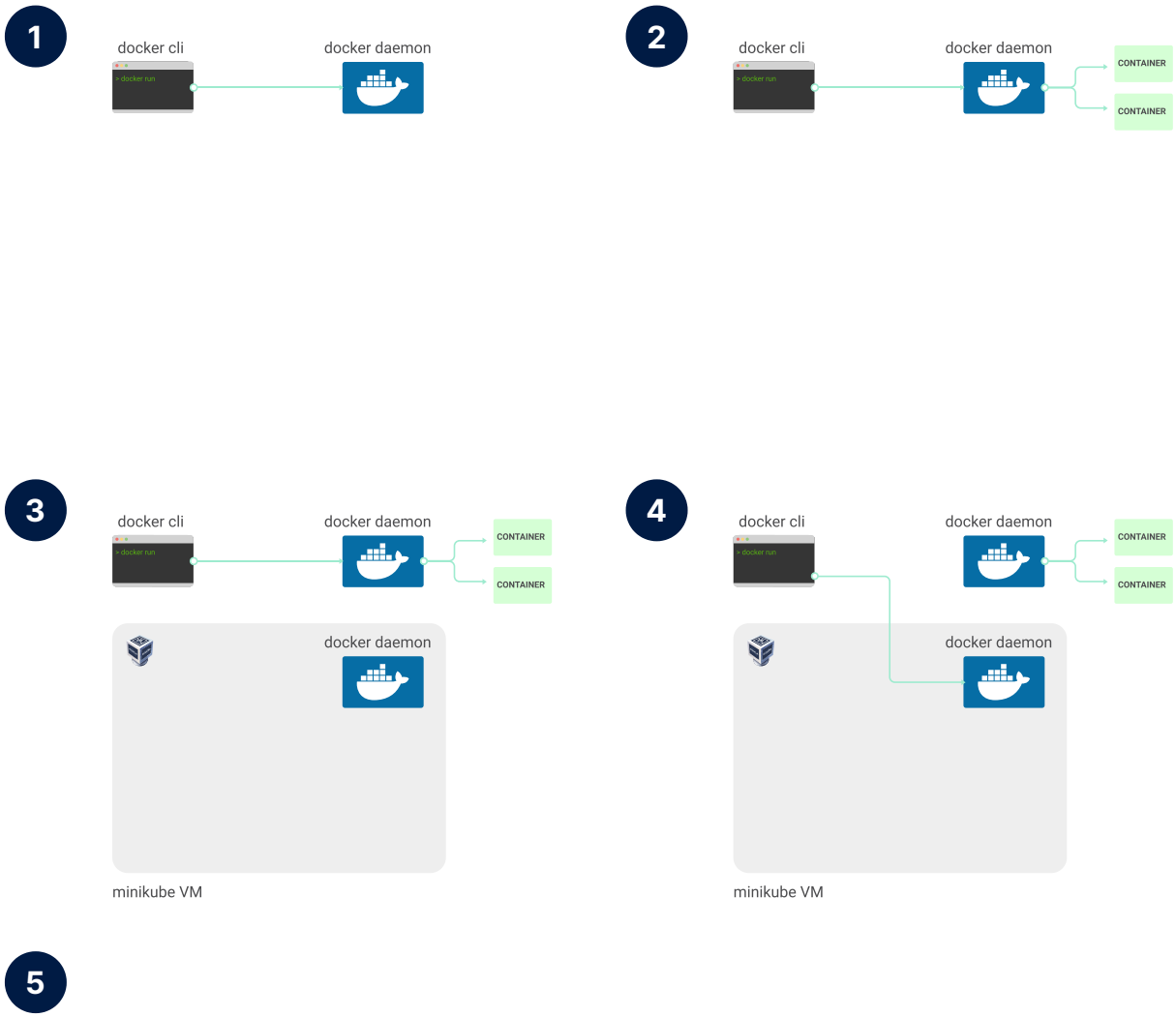


**Fig. 1** In Docker's client-server architecture you have a client sending commands to a server.

**Fig. 2** When you create containers on your computer, you normally send instructions to the local Docker daemon.

**Fig. 3** `minikube` is a local Kubernetes cluster that is packaged as a virtual machine. It features its own Docker daemon.

**Fig. 4** Instead of building, uploading and downloading the images to the `minikube`'s Docker daemon, you could build the images directly on `minikube`.

**Fig. 5** Every time you build and image or run a container, those are created inside the `minikube` virtual machine.

You can connect your Docker CLI to the remote Docker demon inside minikube with:

INSTRUCTIONS FOR SHELL

bash

```
$ eval $(minikube -p minikube docker-env)
```

INSTRUCTIONS FOR POWERSHELL

PowerShell

```
PS> & minikube -p minikube docker-env | Invoke-Expression
```

INSTRUCTIONS FOR CMD.EXE

PowerShell

```
PS> REM @FOR /f "tokens=*" %i IN ('minikube -p minikube docker-env') DO @%
```

> Please note that you should evaluate that command every time you open a new terminal window and wish to connect to the remote Docker daemon.

When you're connected, you can verify that the Docker daemon has a few containers running:

bash

```
$ docker ps
CONTAINER ID   IMAGE                  COMMAND
c0a09c8dcb24   ba04bb24b957           "/storage-provisioner"
2c808ad27e8e   db91994f4ee8           "/coredns -conf /etc…"
d47bbd039ab0   788e63d07298           "/usr/local/bin/kube…"
87a4be9b8bbf   k8s.gcr.io/pause:3.2   "/pause"
14a19b073040   k8s.gcr.io/pause:3.2   "/pause"
3fb274f7889d   k8s.gcr.io/pause:3.2   "/pause"
ee124aac2c1c   60d957e44ec8           "kube-scheduler --au…"
72080d7884b1   95d99817fc33           "kube-apiserver --ad…"
21ef0b1bc69a   3a1a2b528610           "kube-controller-man…"
a18906334302   05b738aa1bc6           "etcd --advertise-cl…"
b75a0a16831e   k8s.gcr.io/pause:3.2   "/pause"
7b060488c2da   k8s.gcr.io/pause:3.2   "/pause"
00b7de9b36c0   k8s.gcr.io/pause:3.2   "/pause"
6a729d4b70f1   k8s.gcr.io/pause:3.2   "/pause"
```

> If you don't see as many containers, there might something wrong with the eval command. Or you might have an `alias` for Docker in your shell profile (you can inspect that with `type docker`).

Open a new terminal window type the same command:

```bash
$ docker ps
CONTAINER ID   IMAGE                  COMMAND
```

**Why are they so different?**

The output is different because, in the former terminal session, the Docker daemon is running in the `minikube`'s virtual machine.

However, the latter command ran against your local Docker daemon.

Earlier you ran the following command to build the container image:

```bash
$ docker build -t first-k8s-app .
```

*Which Docker daemon were you connected to?*

Let's test that.

Run the following commands in both terminals

bash

```
$ docker images | grep first-k8s-app
```

*Which Docker demon has the image?*

You built the image against your local Docker daemon.

The image is not available in Minikube.

*So how do you create the image in Minikube?*

With the same command that you used earlier.

However, you should pay attention and run the command in the same terminal window where the Docker CLI is connected to the Minikube:

bash

```
$ docker build -t first-k8s-app .
```

And you can verify that the image was built correctly with:

bash

```
$ docker images | grep first-k8s-app
REPOSITORY      TAG       IMAGE ID       CREATED             SIZE
first-k8s-app   latest    dc2a8fd35e2e   30 seconds ago   165MB
nginx           1.17.9    d9bfca6c7741   2 weeks ago      150MB
```

Both terminal session should have that image now.

# Deploying the app in Kubernetes

The first time you deployed an app in Kubernetes, you used the commands `kubectl run`, and `kubectl expose`.

`kubectl run` created a Pod template, replaced the values you passed as arguments and submitted it to the cluster.

You also learned that:

1. **Creating Pods manually isn't always a good idea.** If the Pod is deleted, there is no one to recreate a new one.
2. If you use `kubectl run` you can't save the YAML resource locally.

Instead of using `kubectl run`, this time, you will create a Deployment definition and use `kubectl apply` to submit the changes.

Let's get started.

Create a Deployment file.

Here is the definition:

app.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
```

```
  replicas: 1
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
        - name: app
          image: first-k8s-app
          ports:
            - containerPort: 80
          imagePullPolicy: IfNotPresent
```

*That looks complicated, but we will break it down and explain it in detail.*

For now, save the above content in a file named `app.yaml`.

**You must be wondering how you can find out about the structure of a Kubernetes resource.**

The answer is, in the Kubernetes API reference.

The Kubernetes API reference contains the specification for every Kubernetes resource, including all the available fields, their data types, default values, required fields, and so on.

Here is the specification of the Deployment resource.

*If you prefer to work in the command-line, there's an even better way.*

The `kubectl explain` command can print the specification of every Kubernetes resource directly in your terminal:

bash

```
$ kubectl explain deployment
KIND:     Deployment
VERSION:  apps/v1

DESCRIPTION:
     Deployment enables declarative updates for Pods and ReplicaSets.
# truncated output
```

The command outputs precisely the same information as the web-based API reference.

To drill down to a specific field use:

bash

```
$ kubectl explain deployment.spec.replicas
KIND:     Deployment
VERSION:  apps/v1

FIELD:    replicas <integer>

DESCRIPTION:
     Number of desired pods. This is a pointer to distinguish between explicit
     zero and not specified. Defaults to 1.
```

**Now that you know how to look up the documentation of Kubernetes resources, let's turn back to the Deployment.**

The first four lines define the type of resource ( `Deployment` ), the version of this resource type ( `apps/v1` ), and the name of this specific resource ( `app` ):

kube/app.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
```

```
spec:
  replicas: 1
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
        - name: app
          image: first-k8s-app
          ports:
            - containerPort: 80
          imagePullPolicy: IfNotPresent
```

Next, you have the desired number of replicas of your Pod: 1.

You don't usually talk about containers in Kubernetes.

Instead, you talk about Pods.

*What is a Pod?*

**A Pod is a wrapper around one or more containers.**

Most often, a Pod contains only a single container — however, for advanced use cases, a Pod may have multiple containers.

If a Pod contains multiple containers, they are treated by Kubernetes as a unit — for example, they are started and stopped together and executed on the same node.

*A Pod is the smallest unit of deployment in Kubernetes — you never work with containers directly, but with Pods that wrap containers.*

Technically, a Pod is a Kubernetes resource, like a Deployment or Service.

**Let's turn back to the Deployment resource.**

The next part ties together the Deployment resource with the Pod replicas:

kube/app.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 1
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
        - name: app
          image: first-k8s-app
          ports:
            - containerPort: 80
          imagePullPolicy: IfNotPresent
```

The `template.metadata:labels` field defines a label for the Pods that wraps your container ( `name: app` ).

The `selector.matchLabels` field selects those Pods with a `name: app` label to belong to this Deployment resource.

> Note that there must be at least one shared label between these two fields.

The next part in the Deployment defines the actual container that you want to run:

kube/app.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  replicas: 1
  selector:
    matchLabels:
      name: app
  template:
    metadata:
      labels:
        name: app
    spec:
      containers:
        - name: app
          image: first-k8s-app
          ports:
            - containerPort: 80
          imagePullPolicy: IfNotPresent
```

It defines the following things:

- A name for the container ( `app` ).
- The name of the Docker image ( `first-k8s-app` ).
- The port that the container listens on (80).

The above arguments should look familiar to you: you used similar ones when you ran your app with `docker run` in the previous section.

*That's not a coincidence.*

When you submit a Deployment resource to the cluster, you can imagine Kubernetes executing `docker run` and launching your container in one of the computers.

The container specification also defines an `imagePullPolicy` of `IfNotPresent` — the instruction makes sure that Docker images stored locally have the precedence.

**A Deployment defines how to run an app in the cluster, but it doesn't make it available to other apps.**

*To expose your app, you need a Service.*

However, instead of using `kubectl expose`, this time, you will create a Service YAML definition.

## Defining a Service

A Service resource makes Pods accessible to other Pods or users inside and outside the cluster.

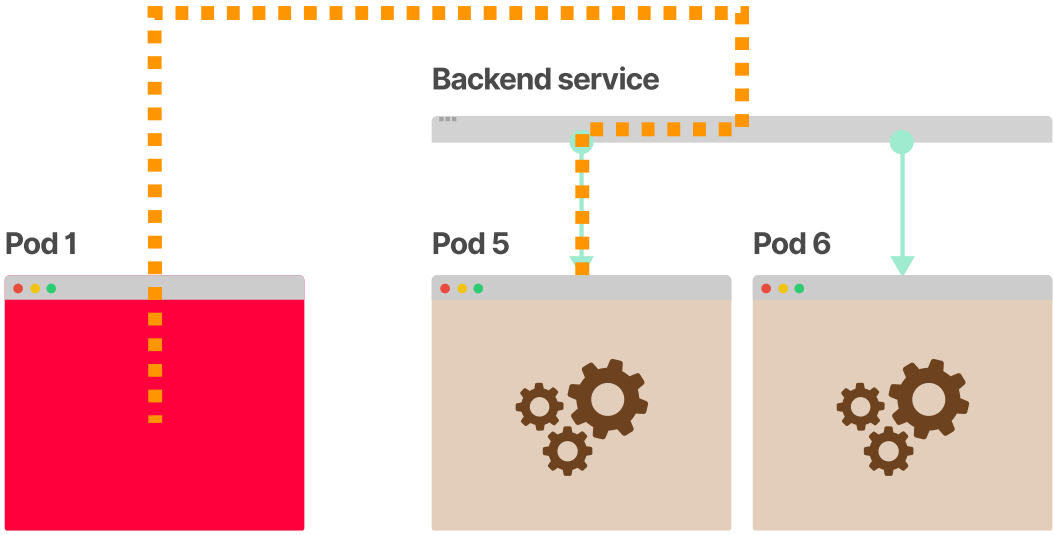



Fig. Services in Kubernetes

In this regard, a Service is akin to a load balancer.
Here is the definition of a Service that makes your Pod accessible to other Pods or users:

app.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
```

```
  name: app
spec:
  selector:
    name: app
  ports:
    - port: 8080
      targetPort: 80
```

> Again, to find out about the available fields of a Service, look it up in the API reference, or, even better, use `kubectl explain service`.

*Where should you save the above definition?*

It is best-practice to save resource definitions that belong to the same application in the same YAML file.

To do so, paste the above content at the beginning of your existing `app.yaml` file, and separate the Service and Deployment resources with three dashes like this:

kube/app.yaml

```
# ... Deployment YAML definition
---
# ... Service YAML definition
```

**Let's break down the Service resource.**

It consists of three crucial parts.

The first part is the selector:

kube/app.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  selector:
    name: app
  ports:
    - port: 8080
      targetPort: 80
```

**It selects the Pods to expose according to their labels.**

In this case, all Pods that have a label of `name: app` will be exposed by the Service.

Note how this label corresponds exactly to what you specified for the Pods in the Deployment resource:

kube/app.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app
spec:
  # ...
  template:
    metadata:
      labels:
        name: app
    # ...
```

It is this label that ties your Service to your Deployment resource.

The next important part is the port:

kube/app.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: app
spec:
  selector:
    name: app
  ports:
    - port: 8080
      targetPort: 80
```

In this case, the Service listens for requests on port 8080 and forwards them to port 80 of the target Pods:

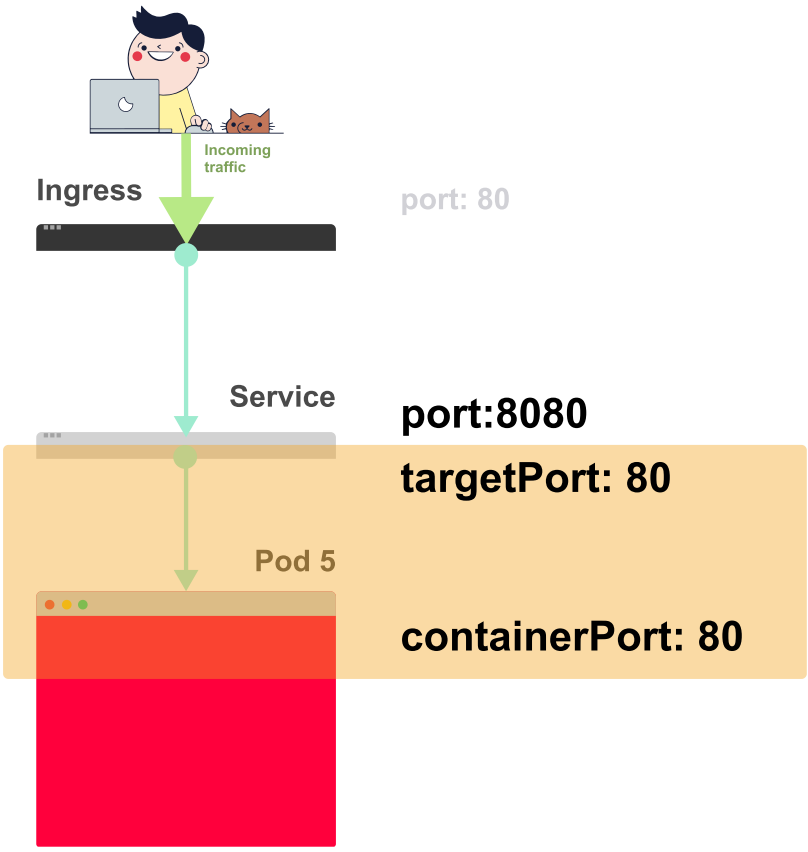



Fig. Service and ports

**Beyond exposing your containers, a Service also ensures continuous availability for your app.**

If one of the Pods crashes and is restarted, the Service makes sure not to route traffic to this container until it is ready again.

Also, when the Pod is restarted, and a new IP address is assigned, the Service automatically handles the update too.

*Furthermore, if you decide to scale your Deployment to 2, 3, 4, or 100 replicas, the Service keeps track of all of these Pods.*

The last resource that you need is the Ingress.

## Defining an Ingress

The Ingress is the component that exposes your Pods to the public internet.

The definition for the resource is similar to what you used in the previous chapter:

app.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - http:
      paths:
      - path: /second
        pathType: Prefix
        backend:
          service:
            name: app
            port:
              number: 8080
```

You can paste the above content at the end of your existing `app.yaml` file, after Deployment and Service like this:

kube/app.yaml

```
# ... Deployment YAML definition
---
# ... Service YAML definition
---
# ... Ingress YAML definition
```

**Let's break down the Ingress resource.**

It consists of three crucial parts.

The first part is the annotation `nginx.ingress.kubernetes.io/rewrite-target: /`.

**Annotations are arbitrary values that you can assign to Kubernetes resources.**

You can create and add your annotations and Kubernetes will gladly accept them.

Annotations are often used to decorate resources with extra arguments.

In this case, the annotation is meant to signal that the base path is rewritten to `/`.

The next part that you should pay attention to is the `spec.rules`.

kube/app.yaml

```
apiVersion: networking.k8s.io/v1
```

```
kind: Ingress
metadata:
  name: app
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - http:
      paths:
      - path: /second
        pathType: Prefix
        backend:
          service:
            name: app
            port:
              number: 8080
```

`spec.rules` is a collection of rules on how the traffic should be routed to your Services.

A single Ingress manifest could have several rules and target different Services.

Each rule can be customised to match:

- A particular path — in this case `/second`.
- The target service — in this case, the Service is `app`, and the port is 8080.
- A hostname — for example, only traffic from `example.com` should go to the Service.

And many more.

Here's a visual recap of your deployment:

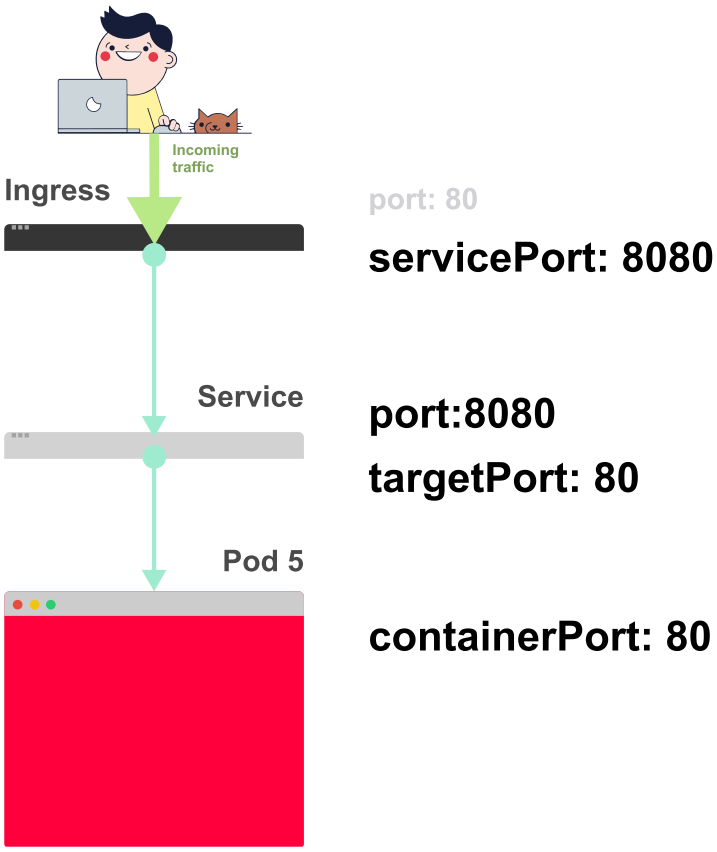



Fig. Ingress, Service and Pod ports

This completes the description of your app — a Deployment, a Service, and an Ingress are all you need.

## Deploying the application

So far, you have created a few YAML files with resource definitions.

*You didn't yet touch the cluster.*

**But now comes the big moment!**

Then submit your resource definitions to Kubernetes with the following command:

bash

```
$ kubectl apply -f app.yaml
deployment.apps/app created
service/app created
ingress.networking.k8s.io/app created
```

As soon as Kubernetes receives your resources, it creates the Pods.

You can watch your Pods coming alive with:

bash

```
$ kubectl get pods --watch
NAME                   READY   STATUS              RESTARTS   AGE
app-5459b555fd-gdbx8   0/1     ContainerCreating   0          36s
```

You should see the Pods transitioning from *Pending* to *ContainerCreating* to *Running*.

*As soon as the Pod is in the Running state, your application is ready.*

Retrieve the IP of Minikube with:

```bash
$ minikube ip
192.168.64.18
```

Visit the URL `http://<replace with minikube ip>/second` in your browser.

**Congratulations!**

You:

1. Wrote a small application.
2. Packaged it as a container.
3. Deployed in Kubernetes.

When you're done testing the app, you can remove it from the cluster with the following command:

```bash
$ kubectl delete -f app.yaml
deployment.apps "app" deleted
service "app" deleted
ingress.networking.k8s.io "app" deleted
namespace "first-k8s-app" deleted
```

The command deletes all the resources that were created by `kubectl apply`.

In this section, you learned how to deploy an application to Kubernetes.

In the next section, you will be practising deploying

applications in Kubernetes.

## Recap

Deploying an application in Kubernetes requires you to:

1. Create the app and package it as a container.
2. If you use Docker, you can define how to build the container image with a `Dockerfile`.
3. Container images are tar archives. You can share them with friends and colleagues with container registries such as Docker Hub.
4. Any Kubernetes cluster can download and run the same images.
5. You deploy the container to Kubernetes using a Deployment, Service and Ingress.
6. You can define the Kubernetes resources in YAML in a single file using `---` as a separator.
7. You can use `kubectl apply` to submit all the resources at once.
8. If you change the context of kubectl, you can deploy the resources into a different Namespace.

Chapter 9

# Organising resources

You might have noticed that the Pod that you created isn't the only Pod in the cluster.

You can list all the Pods in the cluster with:

bash

```
$ kubectl get pods --all-namespaces
NAMESPACE     NAME                               READY   STATUS
kube-system   coredns-74ff55c5b-t444w            1/1     Running
kube-system   etcd-minikube                      1/1     Running
kube-system   kube-apiserver-minikube            1/1     Running
kube-system   kube-controller-manager-minikube   1/1     Running
kube-system   kube-proxy-98q84                   1/1     Running
kube-system   kube-scheduler-minikube            1/1     Running
kube-system   storage-provisioner                1/1     Running
```

*But why are the other Pods not visible by default?*

*Why are you using* `--all-namespaces` *?*

Kubernetes clusters could have hundreds of different Pods and serve dozens of different teams.

**There should be a way to segregate and isolate resources.**

As an example, you might want to group all the resources that belong to the backend API of your website.

Those resources could include:

- Deployments for all the microservices.
- Services.
- Ingress to expose the APIs.

You can create logical sets of resources using a feature called Namespaces.

A Namespace is like a basket where you can store your resources.

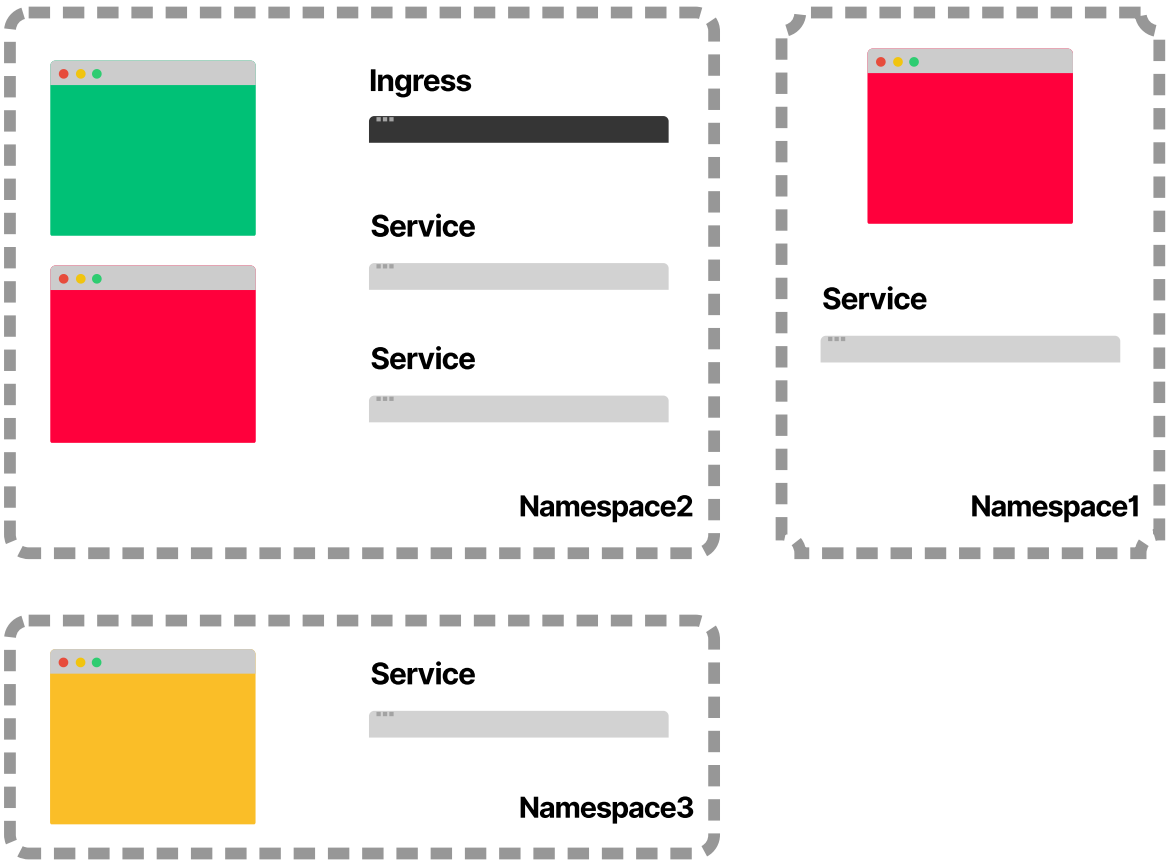



**Fig.** You can group Kubernetes resources in namespaces

When you create, list, update or retrieve a resource, `kubectl` executes the current command in the current Namespace.

If you create a Pod, the Pod is created in the current Namespace.

Unless, of course, you override the request and select a different Namespace:

bash

```
$ kubectl get pods --namespace kube-system
NAME                          READY   STATUS    RESTARTS   AGE
coredns-f9fd979d6-vnqnb       0/1     Running   0          32s
etcd                          0/1     Running   0          24s
kube-apiserver                1/1     Running   0          24s
kube-controller-manager       0/1     Running   0          24s
kube-proxy-m2929              1/1     Running   0          32s
kube-scheduler                0/1     Running   0          24s
storage-provisioner           1/1     Running   0          26s
```

> Please note that you can customise the current Namespace for kubectl by changing your context with `kubectl config set-context --current --namespace=<insert-namespace-name-here>`. If you don't customise it, the Namespace `default` is selected by default.

Kubernetes has a few Namespaces already.
You can list all Namespaces with:

bash

```
$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   107s
kube-node-lease   Active   111s
kube-public       Active
```

```
 111s
kube-system       Active   111s
```

And you can create your Namespace too.
Create a `namespace.yaml` file:

kube/namespace.yaml

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: first-k8s-app
```

You can submit the resource to the cluster with:

bash

```
$ kubectl apply -f namespace.yaml
namespace/first-k8s-app created
```

You can verify that the Namespace was created with:

bash

```
$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   3m1s
first-k8s-app     Active   10s
kube-node-lease   Active   3m5s
kube-public       Active   3m5s
kube-system       Active   3m5s
```

The Namespace `first-k8s-app` should be part of that list.

*Is there any Pod in that Namespace?*
You can check it with:

```
$ kubectl get pods --namespace first-k8s-app
No resources found in first-k8s-app namespace.
```

*No, there isn't.*
But you will create one now!
Let's save the definition for a Pod:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx
```

Then, create the resource in the Namespace `first-k8s-app` with:

bash

```
$ kubectl apply -f pod.yaml --namespace first-k8s-app
pod/nginx created
```

*What happens when you list the Pods?*

bash

```
$ kubectl get pods
NAME                     READY   STATUS    RESTARTS   AGE
```

*The Nginx Pod isn't there!*

Of course, kubectl retrieved the Pods from the `default` Namespace!

You can list the Pods in the Namespace `first-k8s-app` with:

bash

```
$ kubectl get pods --namespace first-k8s-app
NAME                     READY   STATUS    RESTARTS   AGE
nginx                    0/1     Running   0          24s
```

And you should see a single Pod running.

Since you will create all of the resources in the `first-k8s-app` Namespace, it's a good idea to configure your `kubectl` command line to always use that Namespace instead of the `default`.

You can do so with:

bash

```
$ kubectl config set-context --current --namespace=first-k8s-app
Context "minikube" modified.
```

Every subsequent command will run in the `first-k8s-app` namespace.

## Namespaces and (lack of) isolation

*A word of caution about Namespaces.*

**Even if your resources or team are organised in Namespaces, there's no built-in security mechanism.**

Pods can still talk to other Pods even if they are in different Namespaces.

If a Pod abuses the resources of the cluster, all Pods might be affected, even if they are in different namespaces.

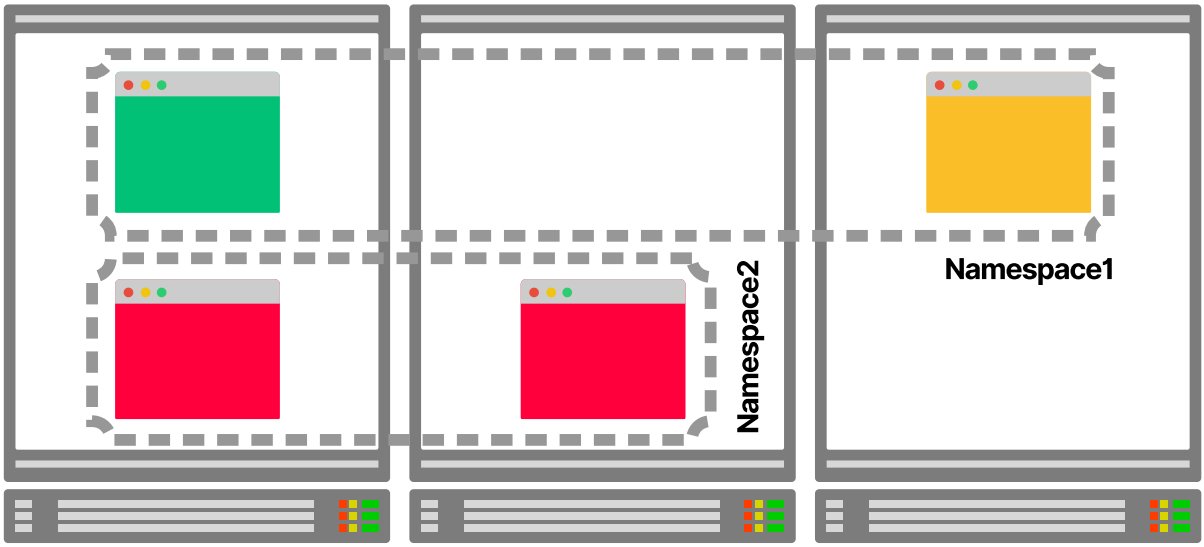



Fig. Namespaces are not designed to isolate workloads

**Namespaces are useful for grouping resources logically.** However, they are not designed to provide rigid boundaries for your workloads.

## Recap

Deploying an application in Kubernetes requires you to:

1. You can use Namespaces to group resources together.
2. If you change the context of kubectl, you can deploy the resources into a different Namespace.
3. Namespaces don't offer any isolation by themselves.

Chapter 10

# Practicing deployments

In the previous chapters, you learned how to:

1. Package an application in a Docker container.
2. Deploy your container with a Deployment
3. Expose your Pods with Services.
4. Expose the Pods to external traffic using an Ingress.
5. Group set of resources using Namespaces

You also learned that commands such as `kubectl run` and `kubectl expose` should be dismissed in favour of YAML files.

It's time to practice what you learned so far and deploy a few more applications.

# Version 2.0.0

You should deploy in Minikube an app that uses Nginx and serves a static HTML file.

The content of the HTML file is:

index.html

```
<html style="background-color:#FABE28">
  <head>
    <title>v2.0.0</title>
  </head>
```

```
  <body style="display:flex;
               align-items:center;
               justify-content:center;
               color:#000000;
               font-family:sans-serif;
               font-size:6rem;
               margin:0;
               letter-spacing:-0.1em">
    <h1>v2.0.0</h1>
  </body>
</html>
```

You should:

- Namespace all resources in Kubernetes.
- Package the application as a Docker container.
- Deploy it in Minikube.
- Scale the Deployment to two replicas.
- Expose the app with a Service.
- Expose the app with an Ingress with path `/v2`.
- Use YAML resource definitions (please don't use `kubectl run`, `kubectl expose` or `kubectl scale`).

In particular, you should create:

- a *Namespace* resource.
- a *Deployment* resource.
- a *Service* resource.
- an *Ingress* resource.

If you aren't sure how to create those resources, you can have a look at the samples below.

## Deployment

You can find the official documentation for Deployments here.
A Deployment looks like this:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-name
  labels:
    key: pod-label
spec:
  selector:
    matchLabels:
      key: pod-label
  template:
    metadata:
      labels:
        key: pod-label
    spec:
      containers:
      - name: container-name
        image: container-image
        ports:
        - containerPort: 3000
```

## Service

You can find the official documentation for Services here.
A Service looks like this:

service.yaml

```yaml
apiVersion: v1
```

```
kind: Service
metadata:
  name: service-name
spec:
  selector:
    key: value
  ports:
    - port: 3000
      targetPort: 9376
```

## Ingress

You can find the official documentation for the Ingress here
An Ingress manifest looks like this:

ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-name
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
  - http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: service-name
            port:
              number: 8888
```

## Namespace

You can find the official documentation for the Namespace

here

A Namespace looks like this:

namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: namespace-name
```

# Version 3.0.0

*How does the deployment change when using an app written in Python?*

Let's assume you created a `python` directory and saved the following HTML file as `index.html`:

index.html

```
<html style="background-color:#00C176">
  <head>
    <title>API</title>
  </head>
  <body style="display:flex;
               align-items:center;
               justify-content:center;
               color:#FFFFFF;
               font-family:sans-serif;
               font-size:6rem;
               margin:0;
               letter-spacing:-0.1em">
```

```
    <h1>API</h1>
  </body>
</html>
```

Python can be used to serve files located in the current directory with:

```bash
$ python3 -m http.server
```

> The command assumes you use Python 3.

The server will be available on http://localhost:8000.

*Can you package the above Python web server and static HTML file as a container?*

*Also, can you create a Deployment?*

You should:

- Package the application as a Docker container. You might need to customise the `Dockerfile` and a different base image.
- Deploy it in Minikube.
- Scale the Deployment to two replicas.
- Expose the app with a Service.
- Expose the app with an Ingress with the domain name

- `k8s-is-great.com`.
- Use YAML resource definitions.
- Group the resources.

Please note that you might need to use curl to test the domain-based routing of the Ingress.

You can use the following command to send a request with a specific domain name:

bash

```
$ curl --header 'Host: k8s-is-great.com' http://<replace with minikube ip>
```

Chapter 11

# Making sense of Deployment, Service and Ingress

When you deployed the application in Kubernetes, you defined three components:

- a **Deployment** — which is a recipe for creating copies of your application called Pods
- a **Service** — an internal load balancer that routes the traffic to Pods
- an **Ingress** — a description of how the traffic should flow from outside the cluster to your Service.

Here's a quick visual recap.

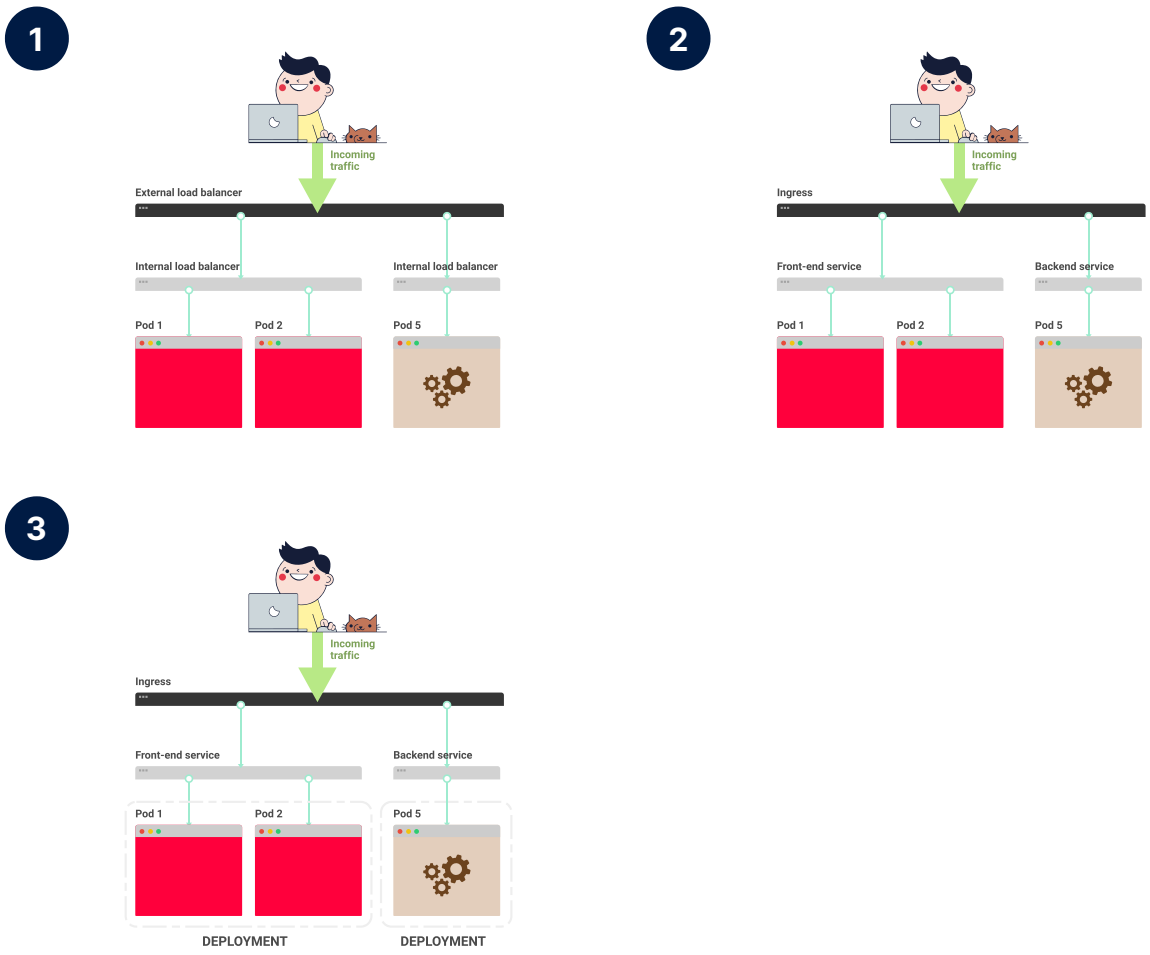


**Fig. 1** In Kubernetes your applications are exposed through two layers of load balancers: internal and external.

**Fig. 2** The internal load balancer is called Service, whereas the external one is called Ingress.

**Fig. 3** Pods are not deployed directly. Instead, the Deployment creates the Pods and whatches over them.

The YAML for such application is similar to this:

hello-world.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-hello-world
  labels:
    track: canary
spec:
  selector:
    matchLabels:
      run: pod-hello-world
  template:
    metadata:
      labels:
        run: pod-hello-world
    spec:
      containers:
      - name: cont1
        image: learnk8s/app:1.0.0
        ports:
        - containerPort: 8080
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: service-hello-world
spec:
  ports:
  - port: 80
    targetPort: 8080
  selector:
    run: pod-hello-world
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-hello-world
spec:
  rules:
  - http:
      paths:
      - path: /
        backend:
          service:
            name: service-hello-world
            port:
              number: 80
        pathType: Prefix
```

You can deploy the above app with:

bash

```
$ kubectl apply -f hello-world.yaml
```

Let's focus on the YAML definition as it's easy to overlook how the components relate to each other.

At this point, you might have a few questions:

- *When should you use port 80 and when port 8080?*

- *Should you create a new port for every Service so that they don't clash?*
- *Do label names matter? Should it be the same everywhere?*

To answer these questions, it's crucial to understand how the three components link to each other.

Let's start with Deployment and Service.

## Connecting Deployment and Service

The surprising news is that Service and Deployment aren't connected at all.

Instead, the Service points to the Pods directly and skips the Deployment altogether.

So what you should pay attention to is how the Pods and the Service are related to each other.

You should remember three things:

1. The Service selector should match at least one label of the Pod
2. The Service `targetPort` should match the `containerPort` of the container inside the Pod

3. The Service `port` can be any number. Multiple Services can use the same port because they have different IP addresses assigned.

The following diagram summarises how to connect the ports:



**Fig. 1** Consider the following Pod exposed by a Service.

**Fig. 2** When you create a Pod, you should define the port `containerPort` for each container in your Pods.

**Fig. 3** When you create a Service, you can define a `port` and a `targetPort`. *But which one should you connect to the container?*

**Fig. 4** `targetPort` and `containerPort` should always match.

**Fig. 5** If your container exposes port 3000, then the `targetPort` should match that number.

If you look at the YAML, the labels and `ports` / `targetPort` should match:

hello-world.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: deployment-hello-world
  labels:
    track: canary
spec:
  selector:
    matchLabels:
      run: pod-hello-world
  template:
    metadata:
      labels:
```

```
        run: pod-hello-world
    spec:
      containers:
      - name: cont1
        image: learnk8s/app:1.0.0
        ports:
        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: service-hello-world
spec:
  ports:
  - port: 80
    targetPort: 8080
  selector:
    run: pod-hello-world
```

*What about the* `track: canary` *label at the top of the Deployment?*

*Should that match too?*

That label belongs to the deployment, and it's not used by the Service's selector to route traffic.

In other words, you can safely remove it or assign it a different value.

*And what about the* `matchLabels` *selector?*

**It always has to match the Pod labels** and it's used by the Deployment to track the Pods.

*Assuming that you made the correct change, how do you test it?*

You can check if the Pods have the right label with the following command:

```bash
$ kubectl get pods --show-labels
```

Or if you have Pods belonging to several applications:

```bash
$ kubectl get pods --selector run=pod-hello-world --show-labels
```

Where `run=pod-hello-world` is the label `run: pod-hello-world`.

*Still having issues?*

You can also connect to the Pod!

You can use the `port-forward` command in kubectl to connect to the Service and test the connection.

```bash
$ kubectl port-forward service/service-hello-world 3000:80
```

Where:

- `service/service-hello-world` is the name of the service
- 3000 is the port that you wish to open on your computer
- 80 is the port exposed by the Service in the `port` field

If you can connect, the setup is correct.
If you can't, you most likely misplaced a label or the port doesn't match.

## Connecting Service and Ingress

The next step in exposing your app is to configure the Ingress.
The Ingress has to know how to retrieve the Service to then retrieve the Pods and route traffic to them.
The Ingress retrieves the right Service by name and port.
Two things should match in the Ingress and Service:

1. The `servicePort` of the Ingress should match the `port` of the Service
2. The `serviceName` of the Ingress should match the `name` of the Service

The following diagram summarises how to connect the ports:




**Fig. 1** You already know that the Service expose a `port`.

**Fig. 2** The Ingress has a field called `servicePort`.

**Fig. 3** The Service `port` and the Ingress `servicePort` should always match.

**Fig. 4** If you decide to assign port 80 to the service, you should change `servicePort` to 80 too.

In practice, you should look at these lines:

hello-world.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: service-hello-world
spec:
  ports:
  - port: 80
    targetPort: 8080
  selector:
    run: pod-hello-world
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-hello-world
spec:
  rules:
  - http:
      paths:
      - backend:
          service:
            name: service-hello-world
            port:
              number: 80
        path: /
        pathType: Prefix
```

*How do you test that the Ingress works?*

You can use the same strategy as before with `kubectl port-forward`, but instead of connecting to a service, you should connect to the Ingress controller.

First, retrieve the Pod name for the Ingress controller with:

bash

```
$ kubectl get pods --all-namespaces
NAMESPACE   NAME                                        READY
```

```
 STATUS
kube-system coredns-5644d7b6d9-jn7cq           1/1   Running
kube-system etcd-minikube                      1/1   Running
kube-system kube-apiserver-minikube            1/1   Running
kube-system kube-controller-manager-minikube   1/1   Running
kube-system kube-proxy-zvf2h                   1/1   Running
kube-system kube-scheduler-minikube            1/1   Running
kube-system nginx-ingress-controller-6fc5bcc   1/1   Running
```

Identify the Ingress Pod (which might be in a different Namespace) and describe it to retrieve the port:

bash

```
$ kubectl describe pod nginx-ingress-controller-6fc5bcc \
 --namespace kube-system \
 | grep Ports
Ports:         80/TCP, 443/TCP, 18080/TCP
```

Finally, connect to the Pod:

bash

```
$ kubectl port-forward nginx-ingress-controller-6fc5bcc 3000:80 --namespace kube-system
```

At this point, every time you visit port 3000 on your computer, the request is forwarded to port 80 on the Ingress controller Pod.

If you visit http://localhost:3000, you should find the app serving a web page.

# Recap

Here's a quick recap on what ports and labels should match:

1. The Service selector should match the label of the Pod
2. The Service `targetPort` should match the `containerPort` of the container inside the Pod
3. The Service port can be any number. Multiple Services can use the same port because they have different IP addresses assigned.
4. The `servicePort` of the Ingress should match the `port` in the Service
5. The name of the Service should match the field `serviceName` in the Ingress

Chapter 12

# Why Pods?

You might wonder why Kubernetes containers are called Pods.

A Pod isn't an application or a container.

## Pods as a collection of containers

A Pod is a collection of containers that work as a single unit.

Those containers cannot exist in isolation, and it makes sense to keep them together.

As an example, you might want to log all incoming requests to your web app.

Or you might want to use a proxy to filter all the request to your app.

You could have one container such as Nginx dedicated to filtering traffic before it reaches the app.

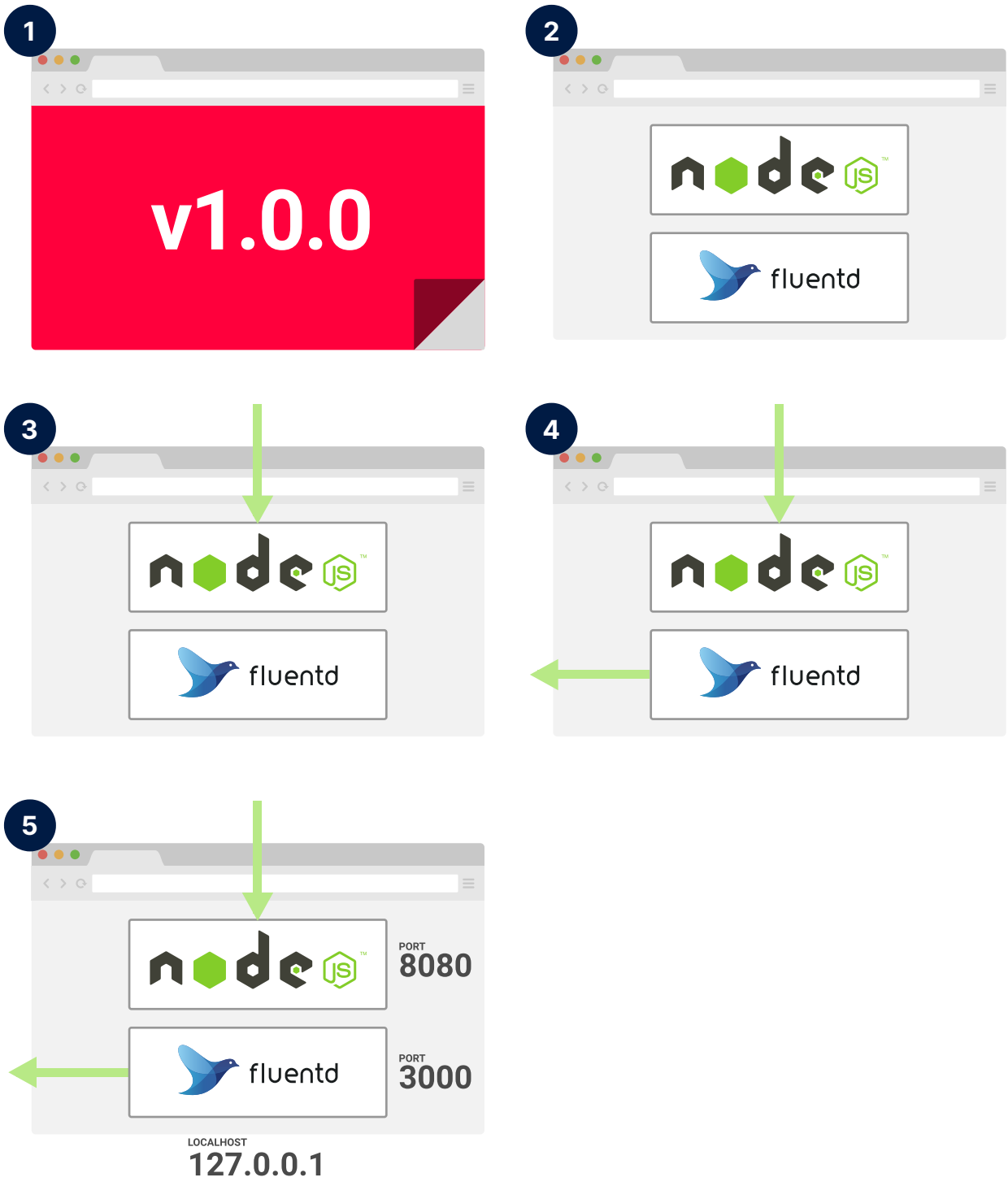



**Fig. 1** You could inspect a Pod and realise that it's made of multiple containers.

**Fig. 2** The Pod above is a combination of Node.js and Fluentd.

**Fig. 3** How do you send all the logs from the Node.js process to the central log aggregator?

**Fig. 4** You could have companion container that collects and ships the logs.

**Fig. 5** Containers grouped into a Pod share the same network as if you were to start them locally on your laptop. You have to pay attention not to use the same port twice.

Applications within a Pod share the volumes too.
When you mount a volume and write a file to disk, you can see the changes from all the other containers inside the same Pod.